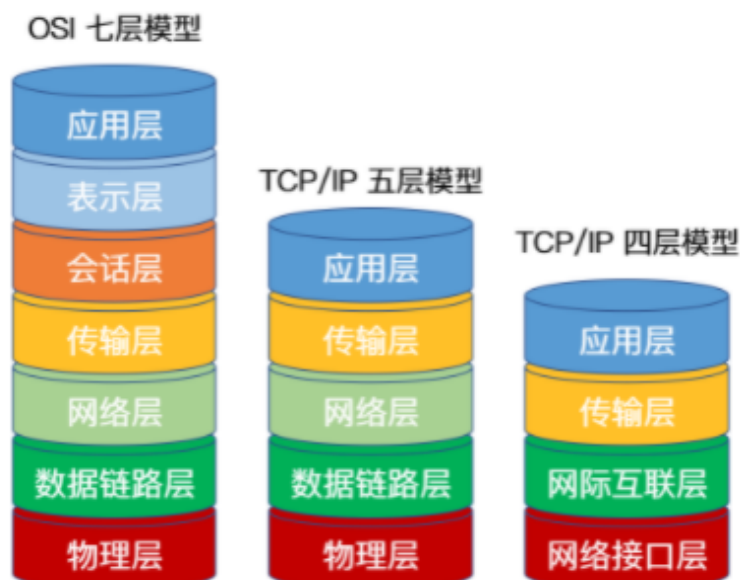


第一部分：协议层次以及它们的服务类型

OSI 七层模型



OSI 模型全称为开放式通信系统互连参考模型，是国际标准化组织（ISO）提出的一个试图使各种计算机在世界范围内互连为网络的标准框架。OSI 将计算机网络体系结构划分为七层，每一层实现各自的功能和协议，并完成与相邻层的接口通信。OSI 的服务定义详细说明了各层所提供的服务。某一层的服务就是该层及其下各层的一种能力，它通过接口提供给更高一层。各层所提供的服务与这些服务是怎么实现的无关。

① 应用层

应用层位于 OSI 参考模型的第七层，其作用是通过应用程序间的交互来完成特定的网络应用。该层协议定义了应用进程之间的交互规则，通过不同的应用层协议为不同的网络应用提供服务。例如域名系统 DNS，支持万维网应用的 HTTP 协议，电子邮件系统采用的 SMTP 协议等。在应用层交互的数据单元我们称之为报文。

② 表示层

表示层的作用是使通信的应用程序能够解释交换数据的含义，其位于 OSI 参考模型的第六层，向上为应用层提供服务，向下接收来自会话层的服务。该层提供的服务主要包括数据压缩，数据加密以及数据描述。这使得应用程序不必担心在各台计算机中表示和存储的内部格式差异。

③ 会话层

会话层就是负责建立、管理和终止表示层实体之间的通信会话。该层提供了数据交换的定界和同步功能，包括了建立检查点和恢复方案的方法。

④ 传输层

传输层的主要任务是为两台主机进程之间的通信提供服务。应用程序利用该服务传送应用层报文。该服务并不针对某一特定的应用，多种应用可以使用同一个传输层服务。由于一台主机可同时运行多个线程，因此传输层有复用和分用的功能。所谓复用就是指多个应用层进程可同时使用下面传输层的服务，分用和复用相反，是传输层把收到的信息分别交付上面应用层中的相应进程。

⑤ 网络层

两台计算机之间传送数据时其通信链路往往不止一条，所传输的信息甚至可能经过很多通信子网。网络层的主要任务就是选择合适的网间路由和交换节点，确保数据按时成功传送。在发送数据时，网络层把传输层产生的报文或用户数据报封装成分组和包向下传输到数据链路层。在网络层使用的协议是无连接的网际协议（Internet Protocol）和许多路由协议，因此我们通常把该层简单地称为 IP 层。

⑥ 数据链路层

数据链路层通常也叫做链路层，在物理层和网络层之间。两台主机之间的数据传输，总是在一段一段的链路上传送的，这就需要使用专门的链路层协议。在两个相邻节点之间传送数据时，数据链路层将网络层交下来的 IP 数据报组装成帧，在两个相邻节点间的链路上传送帧。每一帧包括数据和必要的控制信息。通过控制信息我们可以知道一个帧的起止比特位置，此外，也能使接收端检测出所收到的帧有无差错，如果发现差错，数据链路层能够简单的丢弃掉这个帧，以避免继续占用网络资源。

⑦ 物理层

作为 OSI 参考模型中最低的一层，物理层的作用是实现计算机节点之间比特流的透明传送，尽可能屏蔽掉具体传输介质和物理设备的差异。使其上面的数据链路层不必考虑网络的具体传输介质是什么。该层的主要任务是确定与传输媒体的接口的一些特性（机械特性、电气特性、功能特性，过程特性）。

TCP/IP 参考模型

OSI 七层模型在提出时的出发点是基于标准化的考虑，而没有考虑到具体的市场需求，使得该模型结构复杂，部分功能冗余，因而完全实现 OSI 参考模型的系统不多。而 TCP/IP 参考模型直接面向市场需求，实现起来也比较容易，因此在一经提出便得到了广泛的应用。基于 TCP/IP 的参考模型将协议分成四个层次，如上图所示，它们分别是：网络访问层、网际互联层、传输层、和应用层。

① 应用层

TCP/IP 模型将 OSI 参考模型中的会话层、表示层和应用层的功能合并到一个应用层实现，通过不同的应用层协议为不同的应用提供服务。例如：FTP、Telnet、DNS、SMTP 等。

② 传输层

该层对应于 OSI 参考模型的传输层，为上层实体提供源端到对端主机的通信功能。传输层定义了两个主要协议：传输控制协议（TCP）和用户数据报协议（UDP）。其中面向连接的 TCP 协议保证了数据的传输可靠性，面向无连接的 UDP 协议能够实现数据包简单、快速地传输。

③ 网际互联层

网际互联层对应 OSI 参考模型的网络层，主要负责相同或不同网络中计算机之间的通信。在网际互联层，IP 协议提供的是一个不可靠、无连接的数据报传递服务。该协议实现两个基本功能：寻址和分段。根据数据报报头中的目的地址将数据传送到目的地址，在这个过程中 IP 负责选择传送路线。除了 IP 协议外，该层另外两个主要协议是互联网组管理协议（IGMP）和互联网控制报文协议（ICMP）。

④ 网络接入层

网络接入层的功能对应于 OSI 参考模型中的物理层和数据链路层，它负责监视数据在主机和网络之间的交换。事实上，TCP/IP 并未真正描述这一层的实现，而由参与互连的各网络使用自己的物理层和数据链路层协议，然后与 TCP/IP 的网络接入层进行连接，因此具体的实现方法将随着网络类型的不同而有所差异。

TCP/IP 五层参考模型

五层体系的协议结构是综合了 OSI 和 TCP/IP 优点的一种协议，包括应用层、传输层、网络层、数据链路层和物理层。其中应用层对应 OSI 的上三层，下四层和 OSI 相同。五层协议的体系结构只是为介绍网络原理而设计的，实际应用还是 TCP/IP 四层体系结构。

OSI 模型和 TCP/IP 模型异同比较

相同点

- ① OSI 参考模型与 TCP/IP 参考模型都采用了层次结构。
- ② 都能够提供面向连接和无连接两种通信服务机制。

不同点

- ① OSI 采用的七层模型； TCP/IP 是四层结构。
- ② TCP/IP 参考模型没有对网络接口层进行细分，只是一些概念性的描述； OSI 参考模型对服务和协议做了明确的区分。
- ③ OSI 先有模型，后有协议规范，适合于描述各种网络； TCP/IP 是先有协议集然后建立模型，不适用于非 TCP/IP 网络。
- ④ TCP/IP 一开始就提出面向连接和无连接服务，而 OSI 一开始只强调面向连接服务，直到很晚才开始制定无连接的服务标准。
- ⑤ OSI 参考模型虽然被看好，但将网络划分为七层，实现起来较困难；相反，TCP/IP 参考模型虽然有许多不尽人意的地方，但作为一种简化的分层结构还是比较成功的。

OSI 和 TCP/IP 协议之间的对应关系

OSI 七层网络模型	TCP/IP 四层概念模型	对应的网络协议
应用层 (Application)	应用层	HTTP, TFTP, FTP, NFS, WAIS, SMTP, Telnet, DNS, SNMP
表示层 (Presentation)		TIFF, GIF, JPEG, PICT
会话层 (Session)		RPC, SQL, NFS, NetBIOS, names, AppleTalk
传输层 (Transport)	传输层	TCP, UDP
网络层 (Network)	网络层	IP, ICMP, ARP, RARP, RIP, IPX
数据链路层 (Data Link)	数据链路层	FDDI, Frame Relay, HDLC, SLIP, PPP
物理层 (Physical)		EIA/TIA-232, EIA/TIA-499, V.35, 802.3

为什么 TCP/IP 去除了表示层和会话层

OSI 参考模型在提出时，他们的理想是非常好的，但实际上，由于会话层、表示层、应用层都是在应用程序内部实现的，最终产出的是一个应用数据包，而应用程序之间是几乎无法实现代码的抽象共享的，这也就造成 OSI 设想中的应用程序维度的分层是无法实现的，例如，我们几乎不会认为数据的压缩、加密算法算是一种协议，而会话的概念则更为抽象，难以用协议来进行描述，所以在后来的 TCP/IP 协议框架的设计中，便将表示层和会话层与应用层整合在一起，让整个过程更为清晰明了。

数据如何在各层之间传输【数据的封装过程】

在发送主机端，一个应用层报文被传送到运输层。在最简单的情况下，运输层收取到报文并附上附加信息，该首部将被接收端的运输层使用。应用层报文和运输层首部信息一道构成了运输层报文段。附加的信息可能包括：允许接收端运输层向上向适当的应用程序交付报文的信息以及差错检测位信息。该信息让接收端能够判断报文中的比特是否在途中已被改变。运输层则向网络层传递该报文段，网络层增加了如源和目的端系统地址等网络层首部信息，生成了网络层数据报。该数据报接下来被传递给链路层，在

数据链路层数据包添加发送端 MAC 地址和接收端 MAC 地址后被封装成数据帧，在物理层数据帧被封装成比特流，之后通过传输介质传送到对端。

第二部分：应用层

HTTP 头部包含哪些信息

HTTP 头部本质上是一个传递额外重要信息的键值对。主要分为：通用头部，请求头部，响应头部和实体头部。

通用头部

协议头	说明	举例
Cache-Control	用来指定当前的请求/回复中是否使用缓存机制	Cache-Control: no-store
Connection	客户端（浏览器）想要优先使用的连接类型	Connection: keep-alive (Upgrade)
Date	报文创建时间	Date: Dec, 26 Dec 2015 17: 30: 00 GMT
Trailer	会实现说明在报文主体后记录哪些首部字段，该首部字段可以使用在 HTTP/1.1 版本分块传输编码时	Trailer: Expires
Transfer-Encoding	用来改变报文格式	Transfer-Encoding: chunked
Upgrade	要求服务器升级到一个高版本协议	Upgrade: HTTP/2.0, SHTTP/1.3, IRC/6.9, RTA/x11
Via	告诉服务器，这个请求是由哪些代理发出的	Via: 1.0 fred, 1.1 itbilu.com.com (Apache/1.1)
Warning	一个一般性的警告，表示在实体内容中可能存在错误	Warning: 199 Miscellaneous warning

请求头部

协议头	说明	举例
Accept	告诉服务器自己允许哪些媒体类型	Accept: text/plain
Accept-Charset	浏览器申明可接受的字符集	Accept-Charset: utf-8
Accept-Encoding	浏览器申明自己接收的编码方法	Accept-Encoding: gzip, deflate
Accept-Language	浏览器可接受的响应内容语言列表	Accept-Language: en-US
Authorization	用于表示 HTTP 协议中需要认证资源的认证信息	Authorization: Basic OSdjJGRpbjpvGVulANlc2SdDE==
Expect	表示客户端要求服务器做出特定的行为	Expect: 100-continue
From	发起此请求的用户的邮件地址	From: user@itbilu.com
Host	表示服务器的域名以及服务器所监听的端口号	Host: www.itbilu.com:80
If-XXX	条件请求	If-Modified-Since: Dec, 26 Dec 2015 17:30:00 GMT
Max-Forwards	限制该消息可被代理及网关转发的次数	Max-Forwards: 10
Range	表示请求某个实体的一部分，字节偏移以 0 开始	Range: bytes=500-999
Referer	表示浏览器所访问的前一个页面，可以认为是之前访问页面的链接将浏览器带到了当前页面	Referer: http://itbilu.com/nodejs
User-Agent	浏览器的身份标识字符串	User-Agent: Mozilla/.....

响应头部

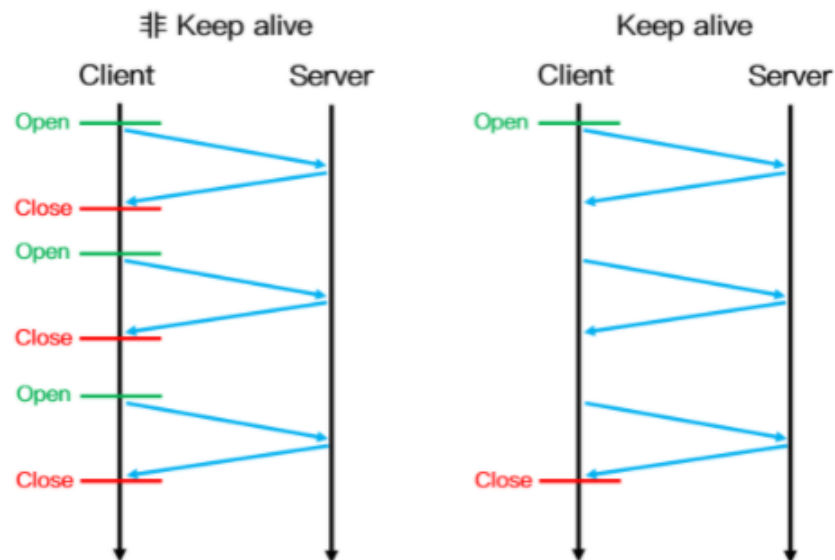
协议头	说明	举例
Accept-Ranges	字段的值表示可用于定义范围的单位	Accept-Ranges: bytes
Age	创建响应的时间	Age: 5744337
ETag	唯一标识分配的资源	Etag: W/"585cd998-7c0f"
Location	表示重定向后的 URL	Location: http://www.zcmhi.com/archives/94.html
Retry-After	告知客户端多久后再发送请求	Retry-After: 120
Server	告知客户端服务器信息	Server: Apache/1.3.27 (Unix) (Red-Hat/Linux)
Vary	缓存控制	Vary: Origin

实体头部

协议头	说明	举例
Allow	对某网络资源的有效的请求行为，不允许则返回405	Allow: GET, HEAD
Content-encoding	返回内容的编码方式	Content-Encoding: gzip
Content-Length	返回内容的字节长度	Content-Length: 348
Content-Language	响应体的语言	Content-Language: en,zh
Content-Location	请求资源可替代的备用的另一地址	Content-Location: /index.htm
Content-MD5	返回资源的MD5校验值	Content-MD5: Q2hIY2sgSW50ZWdyaXR5IQ==
Content-Range	在整个返回体中本部分的字节位置	Content-Range: bytes 21010-47021/47022
Content-Type	返回内容的MIME类型	Content-Type: text/html; charset=utf-8
Expires	响应过期的日期和时间	Expires: Thu, 01 Dec 2010 16:00:00 GMT
Last-Modified	请求资源的最后修改时间	Last-Modified: Tue, 15 Nov 2010 12:45:26 GMT

Keep-Alive 和非 Keep-Alive 区别，对服务器性能有影响吗

在早期的 HTTP/1.0 中，浏览器每次发起 HTTP 请求都要与服务器创建一个新的 TCP 连接，服务器完成请求处理后立即断开 TCP 连接，服务器不跟踪每个客户也不记录过去的请求。然而创建和关闭连接的过程需要消耗资源和时间，为了减少资源消耗，缩短响应时间，就需要重用连接。在 HTTP/1.1 版本中默认使用持久连接，在此之前的 HTTP 版本的默认连接都是使用非持久连接，如果想要在旧版本的 HTTP 协议上维持持久连接，则需要指定 connection 的首部字段的值为 Keep-Alive 来告诉对方这个请求响应完成后不要关闭，下一次咱们还用这个请求继续交流，我们用一个示意图来更加生动的表示两者的区别：



对于非 Keep-Alive 来说，必须为每一个请求的对象建立和维护一个全新的连接。对于每一个这样的连接，客户机和服务器都要分配 TCP 的缓冲区和变量，这给服务器带来的严重的负担，因为一台 Web 服务器可能同时服务于数以百计的客户机请求。在 Keep-Alive 方式下，服务器在响应后保持该 TCP 连接打开，在同一个客户机与服务器之间的后续请求和响应报文可通过相同的连接进行传送。甚至位于同一台服务器的多个 Web 页面在从该服务器发送给同一个客户机时，可以在单个持久 TCP 连接上进行。

然而，Keep-Alive 并不是没有缺点的，当长时间的保持 TCP 连接时容易导致系统资源被无效占用，若对 Keep-Alive 模式配置不当，将有可能比非 Keep-Alive 模式带来的损失更大。因此，我们需要正确地设置 keep-alive timeout 参数，当 TCP 连接在传送完最后一个 HTTP 响应，该连接会保持 keepalive_timeout 秒，之后就开始关闭这个链接。

HTTP 长连接短连接使用场景是什么

长连接：多用于操作频繁，点对点的通讯，而且客户端连接数目较少的情况。例如即时通讯、网络游戏等。

短连接：用户数目较多的Web网站的 HTTP 服务一般用短连接。例如京东，淘宝这样的大型网站一般客户端数量达到千万级甚至上亿，若采用长连接势必会使得服务端大量的资源被无效占用，所以一般使用的是短连接。

怎么知道 HTTP 的报文长度

当响应消息中存在 Content-Length 字段时，我们可以直接根据这个值来判断数据是否接收完成，例如客户端向服务器请求一个静态页面或者一张图片时，服务器能够很清楚的知道请求内容的大小，因此可以通过消息首部字段 Content-Length 来告诉客户端需要接收多少数据，但是如果服务器预先不知道请求内容的大小，例如加载动态页面的时候，就需要使用 Transfer-Encoding: chunked 的方式来代替 Content-Length。

分块传输编码（Chunked transfer encoding）是 HTTP/1.1 中引入的一种数据传输机制，其允许 HTTP 由服务器发送给客户端的数据可以分成多个部分，当数据分解成一系列数据块发送时，服务器就可以发送数据而不需要预先知道发送内容的总大小，每一个分块包含十六进制的长度值和数据，最后一个分块长度值为0，表示实体结束，客户机可以以此为标志确认数据已经接收完毕。

HTTP 方法了解哪些

HTTP/1.0 定义了三种请求方法：GET, POST 和 HEAD 方法。

HTTP/1.1 增加了六种请求方法：OPTIONS, PUT, PATCH, DELETE, TRACE 和 CONNECT 方法。

方法	描述
GET	请求指定的页面信息，并返回具体内容，通常只用于读取数据。
HEAD	类似于 GET 请求，只不过返回的响应中没有具体的内容，用于获取报头。
POST	向指定资源提交数据进行处理请求（例如提交表单或者上传文件）。数据被包含在请求体中。POST 请求可能会导致新的资源的建立或已有资源的更改。
PUT	替换指定的资源，没有的话就新增。
DELETE	请求服务器删除 URL 标识的资源数据。
CONNECT	将服务器作为代理，让服务器代替用户进行访问。
OPTIONS	向服务器发送该方法，会返回对指定资源所支持的 HTTP 请求方法。
TRACE	回显服务器收到的请求数据，即服务器返回自己收到的数据，主要用于测试和诊断。
PATCH	是对 PUT 方法的补充，用来对已知资源进行局部更新。

GET 和 POST 的区别

- get 提交的数据会放在 URL 之后，并且请求参数会被完整的保留在浏览器的记录里，由于参数直接暴露在 URL 中，可能会存在安全问题，因此往往用于获取资源信息。而 post 参数放在请求主体中，并且参数不会被保留，相比 get 方法，post 方法更安全，主要用于修改服务器上的资源。
- get 请求只支持 URL 编码，post 请求支持多种编码格式。
- get 只支持 ASCII 字符格式的参数，而 post 方法没有限制。
- get 提交的数据大小有限制（这里所说的限制是针对浏览器而言的），而 post 方法提交的数据没有限制
- get 方式需要使用 Request.QueryString 来取得变量的值，而 post 方式通过 Request.Form 来获取。
- get 方法产生一个 TCP 数据包，post 方法产生两个（并不是所有的浏览器中都产生两个）。

GET 的长度限制是多少

HTTP 中的 GET 方法是通过 URL 传递数据的，而 URL 本身并没有对数据的长度进行限制，真正限制 GET 长度的是浏览器，例如 IE 浏览器对 URL 的最大限制为 2000 多个字符，大概 2KB 左右，像 Chrome, FireFox 等浏览器能支持的 URL 字符数更多，其中 FireFox 中 URL 最大长度限制为 65536 个字符，Chrome 浏览器中 URL 最大长度限制为 8182 个字符。并且这个长度不是只针对数据部分，而是针对整个 URL 而言，在这之中，不同的服务器同样影响 URL 的最大长度限制。因此对于特定的浏览器，GET 的长度限制不同。

由于 POST 方法请求参数在请求主体中，理论上讲，post 方法是没有大小限制的，而真正起限制作用的是服务器处理程序的处理能力。

HTTP 与 HTTPS 的工作方式【建立连接的过程】

HTTP

HTTP (Hyper Text Transfer Protocol: 超文本传输协议) 是一种简单的请求 - 响应协议, 被用于在 Web 浏览器和网站服务器之间传递消息。HTTP 使用 TCP (而不是 UDP) 作为它的支撑运输层协议。其默认工作在 TCP 协议 80 端口, HTTP 客户机发起一个与服务器的 TCP 连接, 一旦连接建立, 浏览器和服务器进程就可以通过套接字接口访问 TCP。客户机从套接字接口发送 HTTP 请求报文和接收 HTTP 响应报文。类似地, 服务器也是从套接字接口接收 HTTP 请求报文和发送 HTTP 响应报文。其通信内容以明文的方式发送, 不通过任何方式的数据加密。当通信结束时, 客户端与服务器关闭连接。

HTTPS

HTTPS (Hyper Text Transfer Protocol over Secure Socket Layer) 是以安全为目标的 HTTP 协议, 在 HTTP 的基础上通过传输加密和身份认证的方式保证了传输过程的安全性。其工作流程如下:

- ① 客户端发起一个 HTTPS 请求, 并连接到服务器的 443 端口, 发送的信息主要包括自身所支持的算法列表和密钥长度等;
- ② 服务端将自身所支持的所有加密算法与客户端的算法列表进行对比并选择一种支持的加密算法, 然后将它和其它密钥组件一同发送给客户端。
- ③ 服务器向客户端发送一个包含数字证书的报文, 该数字证书中包含证书的颁发机构、过期时间、服务端的公钥等信息。
- ④ 最后服务端发送一个完成报文通知客户端 SSL 的第一阶段已经协商完成。
- ⑤ SSL 第一次协商完成后, 客户端发送一个回应报文, 报文中包含一个客户端生成的随机密码串, 称为 `pre_master_secre`, 并且该报文是经过证书中的公钥加密过的。
- ⑥ 紧接着客户端会发送一个报文提示服务端在此之后的报文是采用 `pre_master_secre` 加密的。
- ⑦ 客户端向服务端发送一个 `finish` 报文, 这次握手中包含第一次握手至今所有报文的整体校验值, 最终协商是否完成取决于服务端能否成功解密。
- ⑧ 服务端同样发送与第 ⑥ 步中相同作用的报文, 已让客户端进行确认, 最后发送 `finish` 报文告诉客户端自己能够正确解密报文。

当服务端和客户端的 `finish` 报文交换完成之后, SSL 连接就算建立完成了, 之后就进行和 HTTP 相同的通信过程, 唯一不同的是在 HTTP 通信过程中并不是采用明文传输, 而是采用对称加密的方式, 其中对称密钥已经在 SSL 的建立过程中协商好了。

HTTPS 和 HTTP 的区别

- HTTP 协议以明文方式发送内容, 数据都是未加密的, 安全性较差。HTTPS 数据传输过程是加密的, 安全性较好。
- HTTP 和 HTTPS 使用的是完全不同的连接方式, 用的端口也不一样, 前者是 80 端口, 后者是 443 端口。
- HTTPS 协议需要到数字认证机构 (Certificate Authority, CA) 申请证书, 一般需要一定的费用。
- HTTP 页面响应比 HTTPS 快, 主要因为 HTTP 使用 3 次握手建立连接, 客户端和服务端需要握手 3 次, 而 HTTPS 除了 TCP 的 3 次握手, 还需要经历一个 SSL 协商过程。

HTTPS 的加密方式

HTTPS 采用对称加密和非对称加密相结合的方式, 首先使用 SSL/TLS 协议进行加密传输, 为了弥补非对称加密的缺点, HTTPS 采用证书来进一步加强非对称加密的安全性, 通过非对称加密, 客户端和服务端协商好之后进行通信传输的对称密钥, 后续的所有信息都通过该对称密钥进行加密解密, 完成整个 HTTPS 的流程。

客户端为什么信任第三方证书

假设中间人篡改了证书原文，由于他没有 CA 机构的私钥，所以无法得到此时加密后的签名，因此无法篡改签名。客户端浏览器收到该证书后会发现原文和签名解密后的值不一致，则说明证书被中间人篡改，证书不可信，从而终止向服务器传输信息。

上述过程说明证书无法被篡改，我们考虑更严重的情况，例如中间人拿到了 CA 机构认证的证书，它想窃取网站 A 发送给客户端的信息，于是它成为中间人拦截到了 A 传给客户端的证书，然后将其替换为自己的证书。此时客户端浏览器收到的是被中间人掉包后的证书，但由于证书里包含了客户端请求的网站信息，因此客户端浏览器只需要把证书里的域名与自己请求的域名比对一下就知道证书有没有被掉包了。

HTTP 是不保存状态的协议,如何保存用户状态

我们知道，假如某个特定的客户机在短时间内两次请求同一个对象，服务器并不会因为刚刚为该用户提供了该对象就不再做出反应，而是重新发送该对象，就像该服务器已经完全忘记不久之前所做过的是一样。因为一个 HTTP 服务器并不保存关于客户机的任何信息，所以我们说 HTTP 是一个无状态协议。

通常有两种解决方案：

① 基于 Session 实现的会话保持

在客户端第一次向服务器发送 HTTP 请求后，服务器会创建一个 Session 对象并将客户端的身份信息以键值对的形式存储下来，然后分配一个会话标识（SessionId）给客户端，这个会话标识一般保存在客户端 Cookie 中，之后每次该浏览器发送 HTTP 请求都会带上 Cookie 中的 SessionId 到服务器，服务器根据会话标识就可以将之前的状态信息与会话联系起来，从而实现会话保持。

优点：安全性高，因为状态信息保存在服务器端。

缺点：由于大型网站往往采用的是分布式服务器，浏览器发送的 HTTP 请求一般要先通过负载均衡器才能到达具体的后台服务器，倘若同一个浏览器两次 HTTP 请求分别落在不同的服务器上时，基于 Session 的方法就不能实现会话保持了。

【解决方法：采用中间件，例如 Redis，我们通过将 Session 的信息存储在 Redis 中，使得每个服务器都可以访问到之前的状态信息】

② 基于 Cookie 实现的会话保持

当服务器发送响应消息时，在 HTTP 响应头中设置 Set-Cookie 字段，用来存储客户端的状态信息。客户端解析出 HTTP 响应头中的字段信息，并根据其生命周期创建不同的 Cookie，这样一来每次浏览器发送 HTTP 请求的时候都会带上 Cookie 字段，从而实现状态保持。基于 Cookie 的会话保持与基于 Session 实现的会话保持最主要的区别是前者完全将会话状态信息存储在浏览器 Cookie 中。

优点：服务器不用保存状态信息，减轻服务器存储压力，同时便于服务端做水平拓展。

缺点：该方式不够安全，因为状态信息存储在客户端，这意味着不能在会话中保存机密数据。除此之外，浏览器每次发起 HTTP 请求时都需要发送额外的 Cookie 到服务器端，会占用更多带宽。

拓展：Cookie被禁用了怎么办？

若遇到 Cookie 被禁用的情况，则可以通过重写 URL 的方式将会话标识放在 URL 的参数里，也可以实现会话保持。

状态码

HTTP 状态码由三个十进制数字组成，第一个数字定义了状态码的类型，后两个并没有起到分类的作用。HTTP 状态码共有 5 种类型：

分类	分类描述
1XX	指示信息--表示请求正在处理
2XX	成功--表示请求已被成功处理完毕
3XX	重定向--要完成的请求需要进行附加操作
4XX	客户端错误--请求有语法错误或者请求无法实现，服务器无法处理请求
5XX	服务器端错误--服务器处理请求出现错误

相应的 HTTP 状态码列表：

状态码	英文名称	中文描述
100	Continue	继续。客户端继续处理请求
101	Switching Protocol	切换协议。服务器根据客户端的请求切换到更高级的协议
200	OK	请求成功。请求所希望的响应头或数据体将随此响应返回
201	Created	请求以实现。并且有一个新的资源已经依据需求而建立
202	Accepted	请求已接受。已经接受请求，但还未处理完成
203	Non-Authoritative Information	非授权信息。请求成功。但返回的 meta 信息不在原始的服务器中，而是一个副本。
204	No Content	无内容。服务器成功处理了请求，但不需要返回任何实体内容
205	Reset Content	重置内容。与 204 类似，不同点是返回此状态码的响应要求请求者重置文档视图
206	Partial Content	部分内容。服务器成功处理了部分 GET 请求
300	Multiple Choices	多种选择。被请求的资源有一系列可供选择的回馈信息，用户或浏览器能够自行选择一个首选地址进行重定向
301	Moved Permanently	永久移动。请求的资源已被永久地移动到新 URI，返回信息会包含新的 URI，浏览器会自动定向到新 URI
302	Found	临时移动。与 301 类似。但资源只是临时被移动，客户端应继续使用原有 URI
303	See Other	查看其它地址。与 301 类似。使用 GET 和 POST 请求查看
		未修改。如果客户端发送了一个带条件的 GET 请求且该请求已被

304	Not Modified	允许，而文档的内容（自上次访问以来或者根据请求的条件）并没有改变，则服务器应当返回这个状态码
305	Use Proxy	使用代理。被请求的资源必须通过指定的代理才能被访问
306	Unused	在最新版的规范中，306状态码已经不再被使用
307	Temporary Redirect	临时重定向。请求的资源现在临时从不同的URI 响应请求，与302类似
400	Bad Request	客户端请求的语法错误，服务器无法理解；请求的参数有误
401	Unauthorized	当前请求需要用户验证
402	Payment Required	该状态码是为了将来可能的需求而预留的
403	Forbidden	服务器已经理解请求，但是拒绝执行它
404	Not Found	请求失败，请求所希望得到的资源未被在服务器上发现
405	Method Not Allowed	客户端请求中的方法被禁止
406	Not Acceptable	请求的资源的内容特性无法满足请求头中的条件，因而无法生成响应实体
407	Proxy Authentication Required	与401响应类似，只不过客户端必须在代理服务器上身份验证
408	Request Time-out	请求超时。服务器等待客户端发送的请求时间过长，超时
409	Conflict	由于和被请求的资源的当前状态之间存在冲突，请求无法完成
410	Gone	被请求的资源在服务器上已经不再可用，而且没有任何已知的转发地址
411	Length Required	服务器拒绝在没有定义 Content-Length 头的情况下接受请求
412	Precondition Failed	客户端请求信息的先决条件错误
413	Request Entity Too Large	服务器拒绝处理当前请求，因为该请求提交的实体数据大小超过了服务器愿意或者能够处理的范围
414	Request-URI Too Large	请求的 URI 长度超过了服务器能够解释的长度，因此服务器拒绝对该请求提供服务
415	Unsupported Media Type	服务器无法处理请求附带的媒体格式

416	Requested range not satisfiable	客户端请求的范围无效
417	Expectation Failed	服务器无法满足Expect的请求头信息
500	Internal Server	服务器遇到了一个未曾预料的情况，导致了它无法完成对请求的处理
501	Not Implemented	服务器不支持当前请求所需要的某个功能
502	Bad Gateway	作为网关或者代理工作的服务器尝试执行请求时，从远程服务器接收到无效的响应
503	Service Unavailable	由于临时的服务器维护或者过载，服务器当前无法处理请求，一段时间后可能恢复正常
504	Gateway Time-out	充当网关或代理的服务器，未及时从远端服务器获取请求
505	HTTP Version not supported	服务器不支持，或者拒绝支持在请求中使用的 HTTP 版本

注：上表基本包含了常见的 HTTP 状态码，而面试时也是围绕以上状态码进行提问，该表并未对每种状态码进行详细的剖析，因此想要深究的同学可以针对不同的状态码自行百度。

面试时针对状态码的常见问法：

① 状态码 301 和 302 的区别？

301：永久移动。请求的资源已被永久的移动到新的URI，旧的地址已经被永久的删除了。返回信息会包括新的URI，浏览器会自动定向到新的URI。今后新的请求都应使用新的URI代替。

302：临时移动。与301类似，客户端拿到服务端的响应消息后会跳转到一个新的 URL 地址。但资源只是临时被移动，旧的地址还在，客户端应继续使用原有URI。

② HTTP 异常状态码知道哪些？

该问题一般只需要回答 3, 4, 5 开头的一些常见异常状态码即可。

HTTP/1.1 和 HTTP/1.0 的区别

主要区别如下：

- 缓存处理：在 HTTP/1.0 中主要使用 header 里的 if-modified-Since, Expires 来做缓存判断的标准。而 HTTP/1.1 请求头中添加了更多与缓存相关的字段，从而支持更为灵活的缓存策略，例如 Entity-tag, If-Unmodified-Since, If-Match, If-None-Match 等可供选择的缓存头来控制缓存策略。
- 节约带宽：当客户端请求某个资源时，HTTP/1.0 默认将该资源相关的整个对象传送给请求方，但很多时候可能客户端并不需要对象的所有信息。而在 HTTP/1.1 的请求头中引入了 range 头域，它允许只请求部分资源，其使得开发者可以多线程请求某一资源，从而充分的利用带宽资源，实现高效并发。
- 错误通知的管理：HTTP/1.1 在 1.0 的基础上新增了 24 个错误状态响应码，例如 414 表示客户端请求中所包含的 URL 地址太长，以至于服务器无法处理；410 表示所请求的资源已经被永久删除。
- Host 请求头：早期 HTTP/1.0 中认为每台服务器都绑定一个唯一的 IP 地址并提供单一的服务，请求消息中的 URL 并没有传递主机名。而随着虚拟主机的出现，一台物理服务器上可以存在多个虚拟主机，并且它们共享同一个 IP 地址。为了支持虚拟主机，HTTP/1.1 中添加了 host 请求头，请

求消息和响应消息中应声明这个字段，若请求消息中缺少该字段时服务端会响应一个 404 错误状态码。

- 长连接：HTTP/1.0 默认浏览器和服务器之间保持短暂连接，浏览器的每次请求都需要与服务器建立一个 TCP 连接，服务器完成后立即断开 TCP 连接。HTTP/1.1 默认使用的是持久连接，其支持在同一个 TCP 请求中传送多个 HTTP 请求和响应。此之前的 HTTP 版本的默认连接都是使用非持久连接，如果想要在旧版本的 HTTP 协议上维持持久连接，则需要指定 Connection 的首部字段的值为 Keep-Alive。

HTTP/1.X 和 HTTP/2.0 的区别

- 相比于 HTTP/1.X 的文本（字符串）传送，HTTP/2.0 采用二进制传送。客户端和服务端传输数据时把数据分成帧，帧组成了数据流，流具有流 ID 标识和优先级，通过优先级以及流依赖能够一定程度上解决关键请求被阻塞的问题。
- HTTP/2.0 支持多路复用。因为流 ID 的存在，通过同一个 HTTP 请求可以实现多个 HTTP 请求传输，客户端和服务端可以通过流 ID 来标识究竟是哪个流从而定位到是哪个 HTTP 请求。
- HTTP/2.0 头部压缩。HTTP/2.0 通过 gzip 和 compress 压缩头部然后再发送，同时通信双方会维护一张头信息表，所有字段都记录在这张表中，在每次 HTTP 传输时只需要传头字段在表中的索引即可，大大减小了重传次数和数据量。
- HTTP/2.0 支持服务器推送。服务器在客户端未经请求许可的情况下，可预先向客户端推送需要的内容，客户端在退出服务时可通过发送复位相关的请求来取消服务端的推送。

HTTP/3 了解吗

HTTP/2 存在的问题

我们知道，传统 Web 平台的数据传输都基于 TCP 协议，而 TCP 协议在创建连接之前不可避免的需要三次握手，如果需提高数据交互的安全性，即增加传输层安全协议（TLS），还会增加更多的握手次数。HTTP 从 1.0 到 2.0，其传输层都是基于 TCP 协议的。即使是带来巨大性能提升的 HTTP/2，也无法完全解决 TCP 协议存在的固有问题（慢启动，拥塞窗口尺寸的设置等）。此外，HTTP/2 多路复用只是减少了连接数，其队头的拥塞问题并没有完全解决，倘若 TCP 丢包率过大，则 HTTP/2 的表现将不如 HTTP/1.1。

QUIC 协议

QUIC（Quick UDP Internet Connections），直译为快速 UDP 网络连接，是谷歌制定的一种基于 UDP 的低延迟传输协议。其主要目的是解决采用传输层 TCP 协议存在的问题，同时满足传输层和应用层对多连接、低延迟等的需求。该协议融合了 TCP、TLS、HTTP/2 等协议的特性，并基于 UDP 传输。该协议带来的主要提升有：

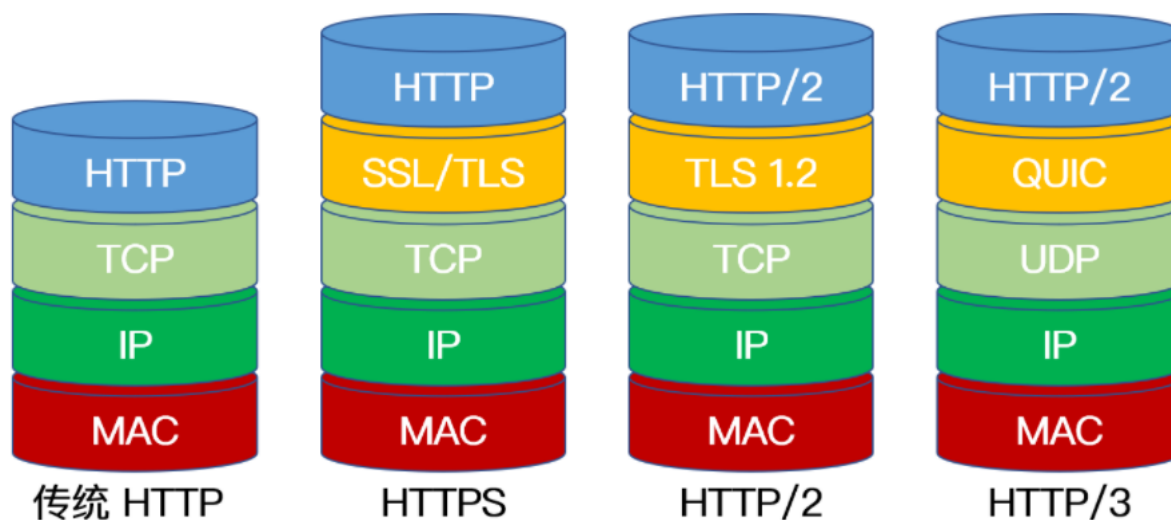
- 低延迟连接。当客户端第一次连接服务器时，QUIC 只需要 1 RTT（Round-Trip Time）延迟就可以建立安全可靠的连接（采用 TLS 1.3 版本），相比于 TCP + TLS 的 3 次 RTT 要更加快捷。之后，客户端可以在本地缓存加密的认证信息，当再次与服务器建立连接时可以实现 0 RTT 的连接建立延迟。
- QUIC 复用了 HTTP/2 协议的多路复用功能，由于 QUIC 基于 UDP，所以也避免了 HTTP/2 存在的队头阻塞问题。
- 基于 UDP 协议的 QUIC 运行在用户域而不是系统内核，这使得 QUIC 协议可以快速的更新和部署，从而很好地解决了 TCP 协议部署及更新的困难。
- QUIC 的报文是经过加密和认证的，除了少量的报文，其它所有的 QUIC 报文头部都经过了认证，报文主体经过了加密。只要有攻击者篡改 QUIC 报文，接收端都能及时发现。
- 具有向前纠错机制，每个数据包携带了除了本身内容外的部分其他数据包的内容，使得在出现少量丢包的情况下，尽量地减少其它包的重传次数，其通过牺牲单个包所携带的有效数据大小换来更少的重传次数，这在丢包数量较小的场景下能够带来一定程度的性能提升。

HTTP/3

HTTP/3 是在 QUIC 基础上发展起来的，其底层使用 UDP 进行数据传输，上层仍然使用 HTTP/2。在 UDP 与 HTTP/2 之间存在一个 QUIC 层，其中 TLS 加密过程在该层进行处理。HTTP/3 主要有以下几个特点：

- ① 使用 UDP 作为传输层进行通信；
- ② 在 UDP 之上的 QUIC 协议保证了 HTTP/3 的安全性。QUIC 在建立连接的过程中就完成了 TLS 加密握手；
- ③ 建立连接快，正常只需要 1 RTT 即可建立连接。如果有缓存之前的会话信息，则直接验证和建立连接，此过程 0 RTT。建立连接时，也可以带有少量业务数据；
- ④ 不和具体底层连接绑定，QUIC 为每个连接的两端分别分配了一个唯一 ID，上层连接只认这对逻辑 ID。网络切换或者断连时，只需要继续发送数据包即可完成连接的建立；
- ⑤ 使用 QPACK 进行头部压缩，因为在 HTTP/2 中的 HPACK 要求传输过程有序，这会导致队头阻塞，而 QPACK 不存在这个问题。

最后我们使用一张图来清晰的表示出 HTTP 协议的发展变化：



DNS 的作用和原理

DNS

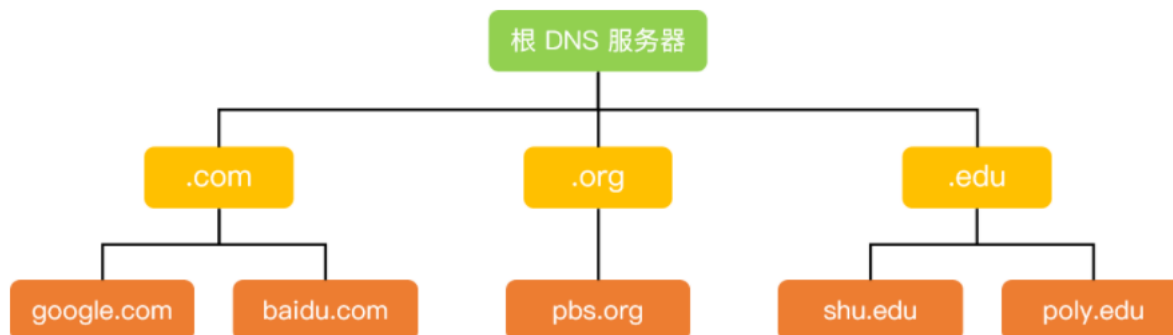
DNS (Domain Name System) 是域名系统的英文缩写，是一种组织成域层次结构的计算机和网络服务命名系统，用于 TCP/IP 网络。

DNS 的作用

通常我们有两种方式识别主机：通过主机名或者 IP 地址。人们喜欢便于记忆的主机名表示，而路由器则喜欢定长的、有着层次结构的 IP 地址。为了满足这些不同的偏好，我们就需要一种能够进行主机名到 IP 地址转换的目录服务，域名系统作为将域名和 IP 地址相互映射的一个分布式数据库，能够使人更方便地访问互联网。

DNS 域名解析原理

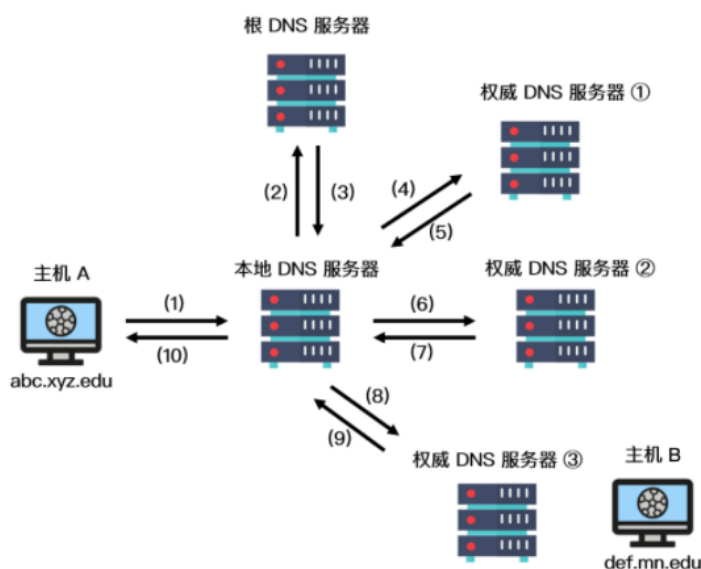
DNS 采用了分布式的设计方案，其域名空间采用一种树形的层次结构：



上图展示了 DNS 服务器的部分层次结构，从上到下依次为根域名服务器、顶级域名服务器和权威域名服务器。其实根域名服务器在因特网上有13个，大部分位于北美洲。第二层为顶级域服务器，这些服务器负责顶级域名（如 com、org、net、edu）和所有国家的顶级域名（如uk、fr、ca 和 jp）。在第三层为权威 DNS 服务器，因特网上具有公共可访问主机（例如 Web 服务器和邮件服务器）的每个组织机构必须提供公共可访问的 DNS 记录，这些记录由组织机构的权威 DNS 服务器负责保存，这些记录将这些主机的名称映射为 IP 地址。

除此之外，还有一类重要的 DNS 服务器，叫做本地 DNS 服务器。本地 DNS 服务器严格来说不在 DNS 服务器的层次结构中，但它对 DNS 层次结构是很重要的。一般来说，每个网络服务提供商（ISP）都有一台本地 DNS 服务器。当主机与某个 ISP 相连时，该 ISP 提供一台主机的 IP 地址，该主机具有一台或多台其本地 DNS 服务器的 IP 地址。主机的本地 DNS 服务器通常和主机距离较近，当主机发起 DNS 请求时，该请求被发送到本地 DNS 服务器，它起着代理的作用，并将该请求转发到 DNS 服务器层次结构中。

我们以一个例子来了解 DNS 的工作原理，假设主机 A（IP 地址为 abc.xyz.edu）想知道主机 B 的 IP 地址（def.mn.edu），如下图所示，主机 A 首先向它的本地 DNS 服务器发送一个 DNS 查询报文。该查询报文含有被转换的主机名 def.mn.edu。本地 DNS 服务器将该报文转发到根 DNS 服务器，根 DNS 服务器注意到查询的 IP 地址前缀为 edu 后向本地 DNS 服务器返回负责 edu 的顶级域名服务器的 IP 地址列表。该本地 DNS 服务器则再次向这些顶级域名服务器发送查询报文。该顶级域名服务器注意到 mn.edu 的前缀，并用权威域名服务器的 IP 地址进行响应。通常情况下，顶级域名服务器并不总是知道每台主机的权威 DNS 服务器的 IP 地址，而只知道中间的某个服务器，该中间 DNS 服务器依次能找到用于相应主机的 IP 地址，我们假设中间经历了权威服务器 ① 和 ②，最后找到了负责 def.mn.edu 的权威 DNS 服务器 ③，之后，本地 DNS 服务器直接向该服务器发送查询报文从而获得主机 B 的 IP 地址。



在上图中，IP 地址的查询其实经历了两种查询方式，分别是递归查询和迭代查询。

拓展：域名解析查询的两种方式

- 递归查询：如果主机所询问的本地域名服务器不知道被查询域名的 IP 地址，那么本地域名服务器就以 DNS 客户端的身份，向其他根域名服务器继续发出查询请求报文，即替主机继续查询，而不是让主机自己进行下一步查询，如上图步骤（1）和（10）。
- 迭代查询：当根域名服务器收到本地域名服务器发出的迭代查询请求报文时，要么给出所要查询的 IP 地址，要么告诉本地服务器下一步应该找哪个域名服务器进行查询，然后让本地服务器进行后续的查询，如上图步骤（2）～（9）。

DNS 为什么用 UDP

更正确的答案是 DNS 既使用 TCP 又使用 UDP。

当进行区域传送（主域名服务器向辅助域名服务器传送变化的那部分数据）时会使用 TCP，因为数据同步传送的数据量比一个请求和应答的数据量要多，而 TCP 允许的报文长度更长，因此为了保证数据的正确性，会使用基于可靠连接的 TCP。

当客户端向 DNS 服务器查询域名（域名解析）的时候，一般返回的内容不会超过 UDP 报文的最大长度，即 512 字节。用 UDP 传输时，不需要经过 TCP 三次握手的过程，从而大大提高了响应速度，但这要求域名解析器和域名服务器都必须自己处理超时和重传从而保证可靠性。

怎么实现 DNS 劫持

DNS 劫持即域名劫持，是通过将原域名对应的 IP 地址进行替换从而使得用户访问到错误的网站或者使得用户无法正常访问网站的一种攻击方式。域名劫持往往只能在特定的网络范围内进行，范围外的 DNS 服务器能够返回正常的 IP 地址。攻击者可以冒充原域名所属机构，通过电子邮件的方式修改组织机构的域名注册信息，或者将域名转让给其它组织，并将新的域名信息保存在所指定的 DNS 服务器中，从而使得用户无法通过对原域名进行解析来访问目的网址。

具体实施步骤如下：

- ① 获取要劫持的域名信息：攻击者首先会访问域名查询站点查询要劫持的域名信息。
- ② 控制域名相应的 E-MAIL 账号：在获取到域名信息后，攻击者通过暴力破解或者专门的方法破解公司注册域名时使用的 E-mail 账号所对应的密码。更高级的攻击者甚至能够直接对 E-mail 进行信息窃取。
- ③ 修改注册信息：当攻击者破解了 E-MAIL 后，会利用相关的更改功能修改该域名的注册信息，包括域名拥有者信息，DNS 服务器信息等。
- ④ 使用 E-MAIL 收发确认函：在修改完注册信息后，攻击者在 E-mail 真正拥有者之前收到修改域名注册信息的相关确认信息，并回复确认修改文件，待网络公司恢复已成功修改信件后，攻击者便成功完成 DNS 劫持。

用户端的一些预防手段：

- 直接通过 IP 地址访问网站，避开 DNS 劫持。
- 由于域名劫持往往只能在特定的网络范围内进行，因此一些高级用户可以通过网络设置让 DNS 指向正常的域名服务器以实现目的网址的正常访问，例如将计算机首选 DNS 服务器的地址固定为 8.8.8.8。

socket() 套接字有哪些

套接字（Socket）是对网络中不同主机上的应用进程之间进行双向通信的端点的抽象，网络进程通信的一端就是一个套接字，不同主机上的进程便是通过套接字发送报文来进行通信。例如 TCP 用主机的 IP 地址 + 端口号作为 TCP 连接的端点，这个端点就叫做套接字。

套接字主要有以下三种类型：

- 流套接字（SOCK_STREAM）：流套接字基于 TCP 传输协议，主要用于提供面向连接、可靠的数据传输服务。由于 TCP 协议的特点，使用流套接字进行通信时能够保证数据无差错、无重复传送，并

按顺序接收，通信双方不需要在程序中进行相应的处理。

- 数据报套接字 (SOCK_DGRAM)：和流套接字不同，数据报套接字基于 UDP 传输协议，对应于无连接的 UDP 服务应用。该服务并不能保证数据传输的可靠性，也无法保证对端能够顺序接收到数据。此外，通信两端不需建立长时间的连接关系，当 UDP 客户端发送一个数据给服务器后，其可以通过同一个套接字给另一个服务器发送数据。当用 UDP 套接字时，丢包等问题需要在程序中进行处理。
- 原始套接字 (SOCK_RAW)：由于流套接字和数据报套接字只能读取 TCP 和 UDP 协议的数据，当需要传送非传输层数据包（例如 Ping 命令时用的 ICMP 协议数据包）或者遇到操作系统无法处理的数据包时，此时就需要建立原始套接字来发送。

URI（统一资源标识符）和 URL（统一资源定位符）之间的区别

URL，即统一资源定位符 (Uniform Resource Locator)，URL 其实就是我们平时上网时输入的网址，它标识一个互联网资源，并指定对其进行操作或获取该资源的方法。例如 <https://leetcode-cn.com/problems/set-all/> 这个 URL，标识一个特定资源并表示该资源的某种形式是可以通过 HTTP 协议从相应位置获得。

从定义即可看出，URL 是 URI 的一个子集，两者都定义了资源是什么，而 URL 还定义了如何能访问到该资源。URI 是一种语义上的抽象概念，可以是绝对的，也可以是相对的，而 URL 则必须提供足够的信息来定位，是绝对的。简单地说，只要能唯一标识资源的就是 URI，在 URI 的基础上给出其资源的访问方式的就 URL。

为什么 fiddler，charles 能抓到你的包【抓取数据包的过程】

假如我们需要抓取客户端的数据包，需要监控客户端与服务器交互之间的网络节点，监控其中任意一个网络节点（网卡），获取所有经过网卡中的数据，对这些数据按照网络协议进行解析，这就是抓包的基本原理。而中间的网络节点不受我们控制，是基本无法实现抓包的，因此只能在客户端与服务器之间进行抓包。

① 当采用抓包工具抓取 HTTP 数据包时，过程较为简单：

- 首先抓包工具会提出代理服务，客户端需要连接该代理；
- 客户端发出 HTTP 请求时，会经过抓包工具的代理，抓包工具将请求的原文进行展示；
- 抓包工具使用该原文将请求发送给服务器；
- 服务器返回结果给抓包工具，抓包工具将返回结果进行展示；
- 抓包工具将服务器返回的结果原样返回给客户端。

这里抓包工具相当于透明人，数据经过的时候它一只手接到数据，然后另一只手把数据传出去。

② 当抓取 HTTPS 数据包时：

- 客户端连接抓包工具提供的代理服务，并安装抓包工具的根证书；
- 客户端发出 HTTPS 请求，抓包工具模拟服务器与客户端进行 TLS 握手交换密钥等流程；
- 抓包工具发送一个 HTTPS 请求给客户端请求的目标服务器，并与目标服务器进行 TLS 握手交换密钥等流程；
- 客户端使用与抓包工具协定好的密钥加密数据后发送给抓包工具；
- 抓包工具使用与客户端协定好的密钥解密数据，并将结果进行展示；
- 抓包工具将解密后的客户端数据，使用与服务器协定好的密钥进行加密后发送给目标服务器；
- 服务器解密数据后，做对应的逻辑处理，然后将返回结果使用与抓包工具协定好的密钥进行加密发送给抓包工具；
- 抓包工具将服务器返回的结果，用与服务器协定好的密钥解密，并将结果进行展示；
- 抓包工具将解密后的服务器返回数据，使用与客户端协定好的密钥进行加密后发送给客户端；
- 客户端解密数据。

这个时候抓包工具对客户端来说相当于服务器，对服务器来说相当于客户端。在这个传输过程中，客户端会以为它就是目标服务器，服务器也会以为它就是请求发起的客户端。

如果你访问一个网站很慢，怎么排查和解决

网页打开速度慢的原因有很多，这里列举出一些较常出现的问题：

- ① 首先最直接的方法是查看本地网络是否正常，可以通过网络测速软件例如电脑管家等对电脑进行测试，若网速正常，我们查看网络带宽是否被占用，例如当你正在下载电影时并且没有限速，是会影响你打开网页的速度的，这种情况往往是处理器内存小导致的；
- ② 当网速测试正常时，我们对网站服务器速度进行排查，通过 ping 命令查看链接到服务器的时间和丢包等情况，一个速度好的机房，首先丢包率不能超过 1%，其次 ping 值要小，最后是 ping 值要稳定，如最大和最小差值过大说明路由不稳定。或者我们也可以查看同台服务器上其他网站的打开速度，看是否其他网站打开也慢。
- ③ 如果网页打开的速度时快时慢，甚至有时候打不开，有可能是空间不稳定的原因。当确定是该问题时，就要找你的空间商解决或换空间商了，如果购买空间的话，可选择购买双线空间或多线空间；如果是在有的地方打开速度快，有的地方打开速度慢，那应该是网络线路的问题。电信线路用户访问放在联通服务器的网站，联通线路用户访问放在电信服务器上的网站，相对来说打开速度肯定是比较慢。
- ④ 从网站本身找原因。网站的问题主要包括网站程序设计、网页设计结构和网页内容三个部分。
 - 网站程序设计：当访问网页中有拖慢网站打开速度的代码，会影响网页的打开速度，例如网页中的统计代码，我们最好将其放在网站的末尾。因此我们需要查看网页程序的设计结构是否合理；
 - 网页设计结构：如果是 table 布局的网站，查看是否嵌套次数太多，或是一个大表格分成多个表格这样的网页布局，此时我们可以采用 div 布局并配合 css 进行优化。
 - 网页内容：查看网页中是否有许多尺寸大的图片或者尺寸大的 flash 存在，我们可以通过降低图片质量，减小图片尺寸，少用大型 flash 加以解决。此外，有的网页可能过多地引用了其他网站的内容，若某些被引用的网站访问速度慢，或者一些页面已经不存在了，打开的速度也会变慢。一种直接的解决方法是去除不必要的加载项。

其他协议

对于应用层来说，考察的重点集中在 HTTP 协议和 DNS 这两块，其他协议考察较少，我们仅加以了解即可。

FTP

- FTP (File Transfer Protocol, 文件传输协议) 是用于在网络上进行文件传输的一套标准协议，使用客户/服务器模式，使用 TCP 数据报，提供交互式访问，双向传输。
- TFTP (Trivial File Transfer Protocol, 简单文件传输协议) 一个小且易实现的文件传输协议，也使用客户/服务器方式，使用 UDP 数据报，只支持文件传输而不支持交互，没有列目录，不能对用户进行身份鉴定。

SMTP

SMTP (Simple Mail Transfer Protocol, 简单邮件传输协议) 是在 Internet 传输 Email 的标准，是一个相对简单的基于文本的协议。在其之上指定了一条消息的一个或多个接收者（在大多数情况下被确认是存在的），然后消息文本会被传输。可以很简单地通过 Telnet 程序来测试一个 SMTP 服务器。SMTP 使用 TCP 端口 25。

DHCP

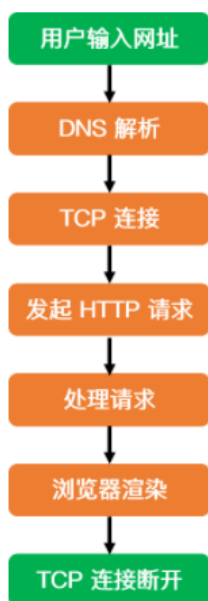
DHCP (Dynamic Host Configuration Protocol, 动态主机设置协议) 是一个局域网的网络协议，使用 UDP 协议工作，主要有两个用途：

- 用于内部网络或网络服务供应商自动分配 IP 地址给用户
- 用于内部网络管理员作为对所有电脑作中央管理的手段

SNMP

SNMP（Simple Network Management Protocol，简单网络管理协议）构成了互联网工程工作小组（IETF，Internet Engineering Task Force）定义的 Internet 协议族的一部分。该协议能够支持网络管理系统，用以监测连接到网络上的设备是否有任何引起管理上关注的情况。

网页解析全过程【用户输入网址到显示对应页面的全过程】



① DNS 解析：当用户输入一个网址并按下回车键的时候，浏览器获得一个域名，而在实际通信过程中，我们需要的是一个 IP 地址，因此我们需要先把域名转换成相应 IP 地址。【具体细节参看问题 16，17】

② TCP 连接：浏览器通过 DNS 获取到 Web 服务器真正的 IP 地址后，便向 Web 服务器发起 TCP 连接请求，通过 TCP 三次握手建立好连接后，浏览器便可以将 HTTP 请求数据发送给服务器了。【三次握手放在传输层详细讲解】

③ 发送 HTTP 请求：浏览器向 Web 服务器发起一个 HTTP 请求，HTTP 协议是建立在 TCP 协议之上的应用层协议，其本质是在建立起的 TCP 连接中，按照 HTTP 协议标准发送一个索要网页的请求。在这一过程中，会涉及到负载均衡等操作。

拓展：什么是负载均衡？

负载均衡，英文名为 Load Balance，其含义是指将负载（工作任务）进行平衡、分摊到多个操作单元上进行运行，例如 FTP 服务器、Web 服务器、企业核心服务器和其他主要任务服务器等，从而协同完成工作任务。负载均衡建立在现有的网络之上，它提供了一种透明且廉价有效的方法扩展服务器和网络设备的带宽、增加吞吐量、加强网络处理能力并提高网络的灵活性和可用性。

负载均衡是分布式系统架构设计中必须考虑的因素之一，例如天猫、京东等大型用户网站中为了处理海量用户发起的请求，其往往采用分布式服务器，并通过引入反向代理等方式将用户请求均匀分发到每个服务器上，而这一过程所实现的就是负载均衡。

④ 处理请求并返回：服务器获取到客户端的 HTTP 请求后，会根据 HTTP 请求中的内容来决定如何获取相应的文件，并将文件发送给浏览器。

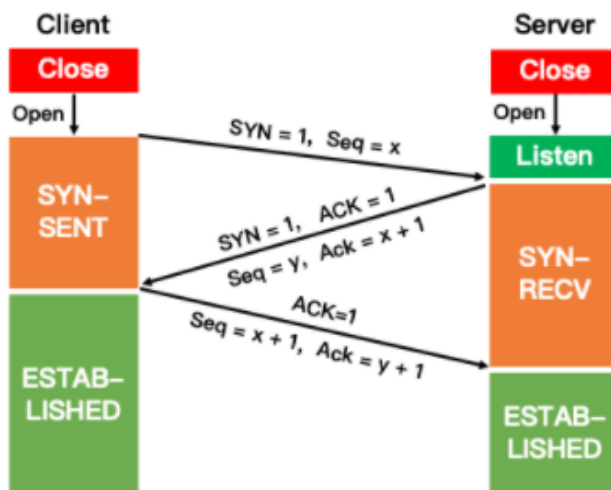
⑤ 浏览器渲染：浏览器根据响应开始显示页面，首先解析 HTML 文件构建 DOM 树，然后解析 CSS 文件构建渲染树，等到渲染树构建完成后，浏览器开始布局渲染树并将其绘制到屏幕上。

⑥ 断开连接：客户端和服务器通过四次挥手终止 TCP 连接。【其中的细节放在传输层详细讲解】

第三部分：传输层

三次握手和四次挥手机制

三次握手



三次握手是 TCP 连接的建立过程。在握手之前，主动打开连接的客户端结束 CLOSE 阶段，被动打开的服务器也结束 CLOSE 阶段，并进入 LISTEN 阶段。随后进入三次握手阶段：

① 首先客户端向服务器发送一个 SYN 包，并等待服务器确认，其中：

- 标志位为 SYN，表示请求建立连接；
- 序号为 $Seq = x$ (x 一般取随机数)；
- 随后客户端进入 SYN-SENT 阶段。

② 服务器接收到客户端发来的 SYN 包后，对该包进行确认后结束 LISTEN 阶段，并返回一段 TCP 报文，其中：

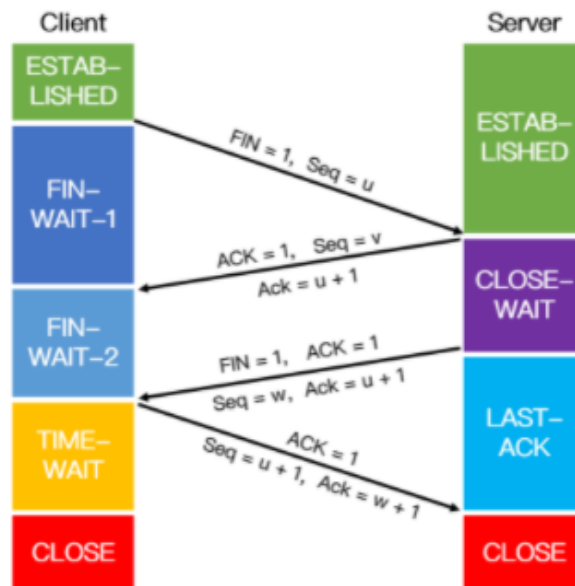
- 标志位为 SYN 和 ACK，表示确认客户端的报文 Seq 序号有效，服务器能正常接收客户端发送的数据，并同意创建新连接；
- 序号为 $Seq = y$ ；
- 确认号为 $Ack = x + 1$ ，表示收到客户端的序号 Seq 并将其值加 1 作为自己确认号 Ack 的值，随后服务器端进入 SYN-RECV 阶段。

③ 客户端接收到发送的 SYN + ACK 包后，明确了从客户端到服务器的数据传输是正常的，从而结束 SYN-SENT 阶段。并返回最后一段报文。其中：

- 标志位为 ACK，表示确认收到服务器端同意连接的信号；
- 序号为 $Seq = x + 1$ ，表示收到服务器端的确认号 Ack，并将其值作为自己的序号值；
- 确认号为 $Ack = y + 1$ ，表示收到服务器端序号 seq，并将其值加 1 作为自己的确认号 Ack 的值。
- 随后客户端进入 ESTABLISHED。

当服务器端收到来自客户端确认收到服务器数据的报文后，得知从服务器到客户端的数据传输是正常的，从而结束 SYN-RECV 阶段，进入 ESTABLISHED 阶段，从而完成三次握手。

四次挥手：



四次挥手即 TCP 连接的释放，这里假设客户端主动释放连接。在挥手之前主动释放连接的客户端结束 ESTABLISHED 阶段，随后开始四次挥手：

① 首先客户端向服务器发送一段 TCP 报文表明其想要释放 TCP 连接，其中：

- 标记位为 FIN，表示请求释放连接；
- 序号为 $\text{Seq} = u$ ；
- 随后客户端进入 FIN-WAIT-1 阶段，即半关闭阶段，并且停止向服务端发送通信数据。

② 服务器接收到客户端请求断开连接的 FIN 报文后，结束 ESTABLISHED 阶段，进入 CLOSE-WAIT 阶段并返回一段 TCP 报文，其中：

- 标记位为 ACK，表示接收到客户端释放连接的请求；
- 序号为 $\text{Seq} = v$ ；
- 确认号为 $\text{Ack} = u + 1$ ，表示是在收到客户端报文的基础上，将其序号值加 1 作为本段报文确认号 Ack 的值；
- 随后服务器开始准备释放服务器端到客户端方向上的连接。

客户端收到服务器发送过来的 TCP 报文后，确认服务器已经收到了客户端连接释放的请求，随后客户端结束 FIN-WAIT-1 阶段，进入 FIN-WAIT-2 阶段。

③ 服务器端在发出 ACK 确认报文后，服务器端会将遗留的待传数据传送给客户端，待传输完成后即经过 CLOSE-WAIT 阶段，便做好了释放服务器端到客户端的连接准备，再次向客户端发出一段 TCP 报文，其中：

- 标记位为 FIN 和 ACK，表示已经准备好释放连接了；
- 序号为 $\text{Seq} = w$ ；
- 确认号 $\text{Ack} = u + 1$ ，表示是在收到客户端报文的基础上，将其序号 Seq 的值加 1 作为本段报文确认号 Ack 的值。

随后服务器端结束 CLOSE-WAIT 阶段，进入 LAST-ACK 阶段。并且停止向客户端发送数据。

④ 客户端收到从服务器发来的 TCP 报文，确认了服务器已经做好释放连接的准备，于是结束 FIN-WAIT-2 阶段，进入 TIME-WAIT 阶段，并向服务器发送一段报文，其中：

- 标记位为 ACK，表示接收到服务器准备好释放连接的信号；
- 序号为 $\text{Seq} = u + 1$ ，表示是在已收到服务器报文的基础上，将其确认号 Ack 值作为本段序号的值；
- 确认号为 $\text{Ack} = w + 1$ ，表示是在收到了服务器报文的基础上，将其序号 Seq 的值作为本段报文确认号的值。

随后客户端开始在 TIME-WAIT 阶段等待 2 MSL。服务器端收到从客户端发出的 TCP 报文之后结束 LAST-ACK 阶段，进入 CLOSED 阶段。由此正式确认关闭服务器端到客户端方向上的连接。客户端等待完 2 MSL 之后，结束 TIME-WAIT 阶段，进入 CLOSED 阶段，由此完成「四次挥手」。

如果三次握手的时候每次握手信息对方没有收到会怎么样

- 若第一次握手服务器未接收到客户端请求建立连接的数据包时，服务器不会进行任何相应的动作，而客户端由于在一段时间内没有收到服务器发来的确认报文，因此会等待一段时间后重新发送 SYN 同步报文，若仍然没有回应，则重复上述过程直到发送次数超过最大重传次数限制后，建立连接的系统调用会返回 -1。
- 若第二次握手客户端未接收到服务器回应的 ACK 报文时，客户端会采取第一次握手失败时的动作，这里不再重复，而服务器端此时将阻塞在 accept() 系统调用处等待 client 再次发送 ACK 报文。
- 若第三次握手服务器未接收到客户端发送过来的 ACK 报文，同样会采取类似于客户端的超时重传机制，若重传次数超过限制后仍然没有回应，则 accept() 系统调用返回 -1，服务器端连接建立失败。但此时客户端认为自己已经连接成功了，因此开始向服务器端发送数据，但是服务器端的 accept() 系统调用已返回，此时没有在监听状态。因此服务器端接收到来自客户端发送来的数据时会发送 RST 报文给客户端，消除客户端单方面建立连接的状态。

为什么要进行三次握手？两次握手可以吗？

三次握手的主要目的是确认自己和对方的发送和接收都是正常的，从而保证了双方能够进行可靠通信。若采用两次握手，当第二次握手后就建立连接的话，此时客户端知道服务器能够正常接收到自己发送的数据，而服务器并不知道客户端是否能够收到自己发送的数据。

我们知道网络往往是非理想状态的（存在丢包和延迟），当客户端发起创建连接的请求时，如果服务器直接创建了这个连接并返回包含 SYN、ACK 和 Seq 等内容的数据包给客户端，这个数据包因为网络传输的原因丢失了，丢失之后客户端就直接收不到返回的数据包。由于客户端可能设置了一个超时时间，一段时间后就关闭了连接建立的请求，再重新发起新的请求，而服务器端是不知道的，如果没有第三次握手告诉服务器客户端能否收到服务器传输的数据的话，服务器端的端口就会一直开着，等到客户端因超时重新发出请求时，服务器就会重新开启一个端口连接。长此以往，这样的端口越来越多，就会造成服务器开销的浪费。

第 2 次握手传回了 ACK，为什么还要传回 SYN

ACK 是为了告诉客户端发来的数据已经接收无误，而传回 SYN 是为了把自己的初始序列号 (Seq) 同步给客户端。

为什么要四次挥手？

释放 TCP 连接时之所以需要四次挥手，是因为 FIN 释放连接报文和 ACK 确认接收报文是分别在两次握手中传输的。当主动方在数据传送结束后发出连接释放的通知，由于被动方可能还有必要的数据要处理，所以会先返回 ACK 确认收到报文。当被动方也没有数据再发送的时候，则发出连接释放通知，对方确认后才完全关闭 TCP 连接。

举个例子：A 和 B 打电话，通话即将结束后，A 说“我没啥要说的了”，B 回答“我知道了”，但是 B 可能还会有要说的话，A 不能要求 B 跟着自己的节奏结束通话，于是 B 可能又巴拉巴拉说了一通，最后 B 说“我说完了”，A 回答“知道了”，这样通话才算结束。

CLOSE-WAIT 和 TIME-WAIT 的状态和意义

在服务器收到客户端关闭连接的请求并告诉客户端自己已经成功收到了该请求之后，服务器进入了 CLOSE-WAIT 状态，然而此时有可能服务端还有一些数据没有传输完成，因此不能立即关闭连接，而 CLOSE-WAIT 状态就是为了保证服务器在关闭连接之前将待发送的数据发送完成。

TIME-WAIT 发生在第四次挥手，当客户端向服务端发送 ACK 确认报文后进入该状态，若取消该状态，即客户端在收到服务端的 FIN 报文后立即关闭连接，此时服务端相应的端口并没有关闭，若客户端在相同的端口立即建立新的连接，则有可能接收到上一次连接中残留的数据包，可能会导致不可预料的异常出现。除此之外，假设客户端最后一次发送的 ACK 包在传输的时候丢失了，由于 TCP 协议的超时重传机制，服务端将重发 FIN 报文，若客户端并没有维持 TIME-WAIT 状态而直接关闭的话，当收到服务端重新发送的 FIN 包时，客户端就会用 RST 包来响应服务端，这将会使得对方认为是有错误发生，然而其实只是正常的关闭连接过程，并没有出现异常情况。

TIME_WAIT 状态会导致什么问题，怎么解决

我们考虑高并发短连接的业务场景，在高并发短连接的 TCP 服务器上，当服务器处理完请求后主动请求关闭连接，这样服务器上会有大量的连接处于 TIME_WAIT 状态，服务器维护每一个连接需要一个 socket，也就是每个连接会占用一个文件描述符，而文件描述符的使用是有上限的，如果持续高并发，会导致一些正常的连接失败。

解决方案：修改配置或设置 SO_REUSEADDR 套接字，使得服务器处于 TIME-WAIT 状态下的端口能够快速回收和重用。

TIME-WAIT 为什么是 2MSL

当客户端发出最后的 ACK 确认报文时，并不能确定服务器端能够收到该段报文。所以客户端在发送完 ACK 确认报文之后，会设置一个时长为 2 MSL 的计时器。MSL (Maximum Segment Lifetime)，指一段 TCP 报文在传输过程中的最大生命周期。2 MSL 即是服务器端发出 FIN 报文和客户端发出的 ACK 确认报文所能保持有效的最大时长。

若服务器在 1 MSL 内没有收到客户端发出的 ACK 确认报文，再次向客户端发出 FIN 报文。如果客户端在 2 MSL 内收到了服务器再次发来的 FIN 报文，说明服务器由于一些原因并没有收到客户端发出的 ACK 确认报文。客户端将再次向服务器发出 ACK 确认报文，并重新开始 2 MSL 的计时。

若客户端在 2MSL 内没有再次收到服务器发送的 FIN 报文，则说明服务器正常接收到客户端 ACK 确认报文，客户端可以进入 CLOSE 阶段，即完成四次挥手。

所以客户端要经历 2 MSL 时长的 TIME-WAIT 阶段，为的是确认服务器能否接收到客户端发出的 ACK 确认报文。

有很多 TIME-WAIT 状态如何解决

服务器可以设置 SO_REUSEADDR 套接字选项来通知内核，如果端口被占用，但 TCP 连接位于 TIME_WAIT 状态时可以重用端口。如果你的服务器程序停止后想立即重启，而新的套接字依旧希望使用同一端口，此时 SO_REUSEADDR 选项就可以避免 TIME-WAIT 状态。

也可以采用长连接的方式减少 TCP 的连接与断开，在长连接的业务中往往不需要考虑 TIME-WAIT 状态，但其实在长连接的业务中并发量一般不会太高。

有很多 CLOSE-WAIT 怎么解决

- 首先检查是不是自己的代码问题（看是否服务端程序忘记关闭连接），如果是，则修改代码。
- 调整系统参数，包括句柄相关参数和 TCP/IP 的参数，一般一个 CLOSE_WAIT 会维持至少 2 个小时的时间，我们可以通过调整参数来缩短这个时间。

TCP 和 UDP 的区别

类型	是否面向连接	传输可靠性	传输形式	传输效率	所需资源	应用场景	首部字节
TCP	是	可靠	字节流	慢	多	文件传输、邮件传输	20~60
UDP	否	不可靠	数据报文段	快	少	即时通讯、域名转换	8个字节

TCP 协议中的定时器

TCP中有七种计时器，分别为：

- 建立连接定时器：顾名思义，该定时器是在建立 TCP 连接的时候使用的，在 TCP 三次握手的过程中，发送方发送 SYN 时，会启动一个定时器（默认为 3 秒），若 SYN 包丢失了，那么 3 秒以后会重新发送 SYN 包，直到达到重传次数。
- 重传定时器：该计时器主要用于 TCP 超时重传机制中，当 TCP 发送报文段时，就会创建特定报文的重传计时器，并可能出现两种情况：
 - ① 若在计时器截止之前发送方收到了接收方的 ACK 报文，则撤销该计时器；
 - ② 若计时器截止时间内并没有收到接收方的 ACK 报文，则发送方重传报文，并将计时器复位。
- 坚持计时器：我们知道 TCP 通过让接受方指明希望从发送方接收的数据字节数（窗口大小）来进行流量控制，当接收端的接收窗口满时，接收端会告诉发送端此时窗口已满，请停止发送数据。此时发送端和接收端的窗口大小均为 0，直到窗口变为非 0 时，接收端将发送一个确认 ACK 告诉发送端可以再次发送数据，但是该报文有可能在传输时丢失。若该 ACK 报文丢失，则双方可能会一直等待下去，为了避免这种死锁情况的发生，发送方使用一个坚持定时器来周期性地向接收方发送探测报文段，以查看接收方窗口是否变大。
- 延迟应答计时器：延迟应答也被称为捎带 ACK，这个定时器是在延迟应答的时候使用的，为了提高网络传输的效率，当服务器接收到客户端的数据后，不是立即回 ACK 给客户端，而是等一段时间，这样如果服务端有数据需要发送给客户端的话，就可以把数据和 ACK 一起发送给客户端了。
- 保活定时器：该定时器是在建立 TCP 连接时指定 SO_KEEPLIVE 时才会生效，当发送方和接收方长时间没有进行数据交互时，该定时器可以用于确定对端是否还活着。
- FIN_WAIT_2 定时器：当主动请求关闭的一方发送 FIN 报文给接收端并且收到其对 FIN 的确认 ACK 后进入 FIN_WAIT_2 状态。如果这个时候因为网络突然断掉、被动关闭的一端宕机等原因，导致请求方没有收到接收方发来的 FIN，主动关闭的一方会一直等待。该定时器的作用就是为了避免这种情况的发生。当该定时器超时的时候，请求关闭方将不再等待，直接释放连接。
- TIME_WAIT 定时器：我们知道在 TCP 四次挥手中，发送方在最后一次挥手之后会进入 TIME_WAIT 状态，不直接进入 CLOSE 状态的主要原因是被动关闭方万一在超时时间内没有收到最后一个 ACK，则会重发最后的 FIN，2 MSL（报文段最大生存时间）等待时间保证了重发的 FIN 会被主动关闭的一方收到且重新发送最后一个 ACK。还有一个原因是在这 2 MSL 的时间段内任何迟到的报文段会被接收方丢弃，从而防止老的 TCP 连接的包在新的 TCP 连接里面出现。

CP 是如何保证可靠性的

- 数据分块：应用数据被分割成 TCP 认为最适合发送的数据块。
- 序列号和确认应答：TCP 给发送的每一个包进行编号，在传输的过程中，每次接收方收到数据后，都会对传输方进行确认应答，即发送 ACK 报文，这个 ACK 报文当中带有对应的确认序列号，告诉发送方成功接收了哪些数据以及下一次的数据从哪里开始发。除此之外，接收方可以根据序列号对数据包进行排序，把有序数据传送给应用层，并丢弃重复的数据。
- 校验和：TCP 将保持它首部和数据部分的校验和。这是一个端到端的校验和，目的是检测数据在传输过程中的任何变化。如果收到报文段的校验和有差错，TCP 将丢弃这个报文段并且不确认收到此

报文段。

- 流量控制：TCP 连接的双方都有一个固定大小的缓冲空间，发送方发送的数据量不能超过接收端缓冲区的大小。当接收方来不及处理发送方的数据，会提示发送方降低发送的速率，防止产生丢包。TCP 通过滑动窗口协议来支持流量控制机制。
- 拥塞控制：当网络某个节点发生拥塞时，减少数据的发送。
- ARQ协议：也是为了实现可靠传输的，它的基本原理就是每发完一个分组就停止发送，等待对方确认。在收到确认后再发下一个分组。
- 超时重传：当 TCP 发出一个报文段后，它启动一个定时器，等待目的端确认收到这个报文段。如果超过某个时间还没有收到确认，将重发这个报文段。

UDP 为什么是不可靠的？bind 和 connect 对于 UDP 的作用是什么

UDP 只有一个 socket 接收缓冲区，没有 socket 发送缓冲区，即只要有数据就发，不管对方是否可以正确接收。而在对方的 socket 接收缓冲区满了之后，新来的数据报无法进入到 socket 接受缓冲区，此数据报就会被丢弃，因此 UDP 不能保证数据能够到达目的地，此外，UDP 也没有流量控制和重传机制，故UDP的数据传输是不可靠的。

和 TCP 建立连接时采用三次握手不同，UDP 中调用 connect 只是把对端的 IP 和 端口号记录下来，并且 UDP 可多次调用 connect 来指定一个新的 IP 和端口号，或者断开旧的 IP 和端口号（通过设置 connect 函数的第二个参数）。和普通的 UDP 相比，调用 connect 的 UDP 会提升效率，并且在高并发服务中会增加系统稳定性。

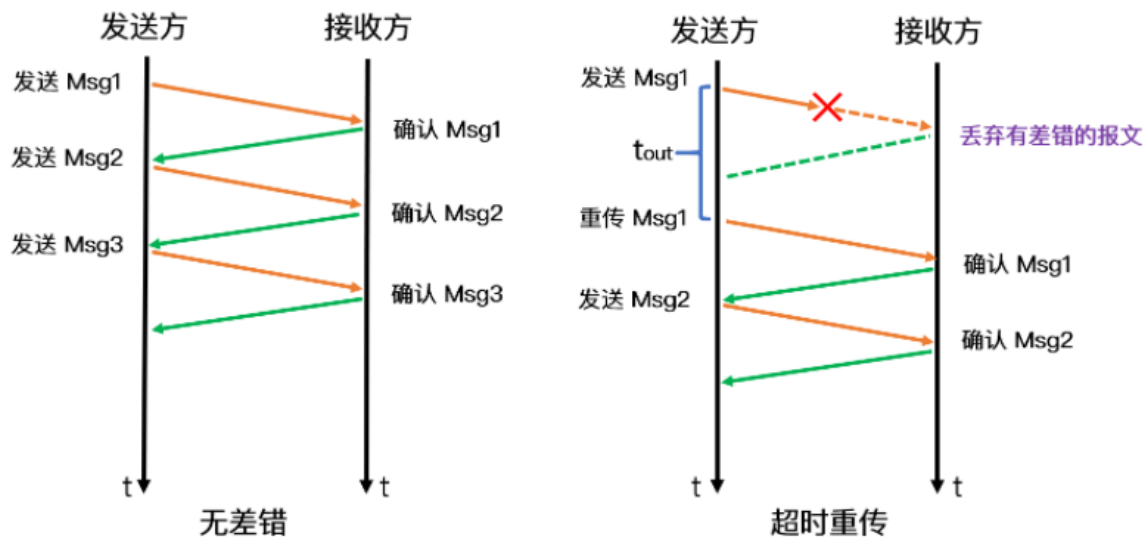
当 UDP 的发送端调用 bind 函数时，就会将这个套接字指定一个端口，若不调用 bind 函数，系统内核会随机分配一个端口给该套接字。当手动绑定时，能够避免内核来执行这一操作，从而在一定程度上提高性能。

TCP 超时重传的原理

发送方在发送一次数据后就开启一个定时器，在一定时间内如果没有得到发送数据包的 ACK 报文，那么就重新发送数据，在达到一定次数还没有成功的话就放弃重传并发送一个复位信号。其中超时时间的计算是超时的核心，而定时时间的确定往往需要进行适当的权衡，因为当定时时间过长会造成网络利用率不高，定时太短会造成多次重传，使得网络阻塞。在 TCP 连接过程中，会参考当前的网络状况从而找到一个合适的超时时间。

TCP 的停止等待协议是什么

停止等待协议是为了实现 TCP 可靠传输而提出的一种相对简单的协议，该协议指的是发送方每发完一组数据后，直到收到接收方的确认信号才继续发送下一组数据。我们通过四种情形来帮助理解停等协议是如何实现可靠传输的：



① 无差错传输

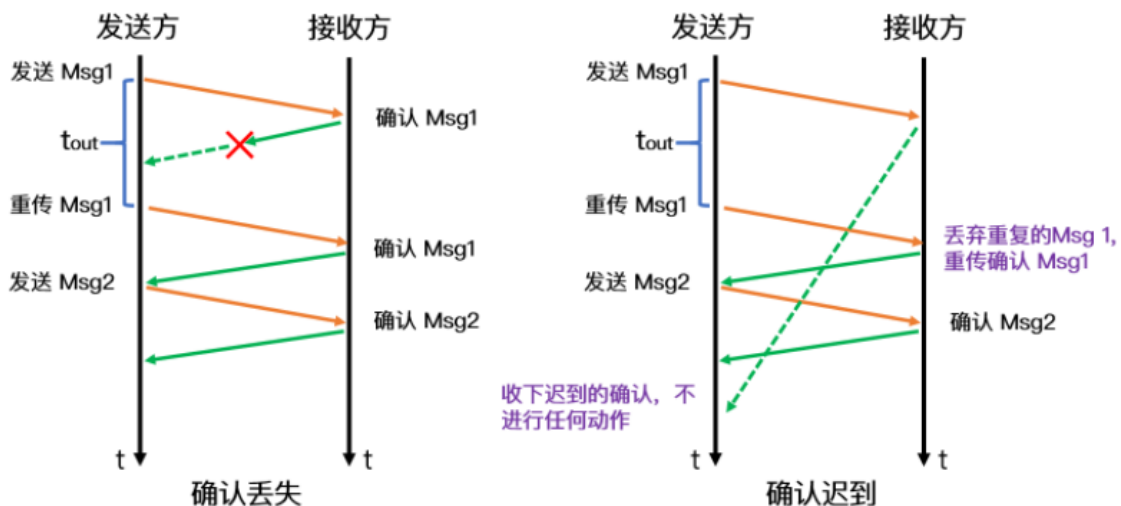
如上述左图所示，A 发送分组 Msg 1，发完就暂停发送，直到收到接收方确认收到 Msg 1 的报文后，继续发送 Msg 2，以此类推，该情形是通信中的一种理想状态。

② 出现差错

如上述右图所示，发送方发送的报文出现差错导致接收方不能正确接收数据，出现差错的情况主要分为两种：

- 发送方发送的 Msg 1 在中途丢失了，接收方完全没收到数据。
- 接收方收到 Msg 1 后检测出现了差错，直接丢弃 Msg 1。

上面两种情形，接收方都不会回任何消息给发送方，此时就会触发超时传输机制，即发送方在等待一段时间后仍然没有收到接收方的确认，就认为刚才发送的数据丢失了，因此重传前面发送过的数据。



③ 确认丢失

当接收方回应的 Msg 1 确认报文在传输过程中丢失，发送方无法接收到确认报文。于是发送方等待一段时间后重传 Msg 1，接收方将收到重复的 Msg1 数据包，此时接收方会丢弃掉这个重复报文并向发送方再次发送 Msg1 的确认报文。

④ 确认迟到

当接收方回应的 Msg 1 确认报文由于网络各种原因导致发送方没有及时收到，此时发送方在超时重传机制的作用下再次发送了 Msg 数据包，接收方此时进行和确认丢失情形下相同的动作（丢弃重复的数据包并再次发送 Msg 1 确认报文）。发送方此时收到了接收方的确认数据包，于是继续进行数据发送。过了一段时间后，发送方收到了迟到的 Msg 1 确认包会直接丢弃。

上述四种情形即停止等待协议中所出现的所有可能情况。

TCP 最大连接数限制

Client 最大 TCP 连接数

client 在每次发起 TCP 连接请求时，如果自己并不指定端口的话，系统会随机选择一个本地端口（local port），该端口是独占的，不能和其他 TCP 连接共享。TCP 端口的数据类型是 unsigned short，因此本地端口个数最大只有 65536，除了端口 0 不能使用外，其他端口在空闲时都可以正常使用，这样可用端口最多有 65535 个。

Server 最大 TCP 连接数

server 通常固定在某个本地端口上监听，等待 client 的连接请求。不考虑地址重用（Unix 的 SO_REUSEADDR 选项）的情况下，即使 server 端有多个 IP，本地监听端口也是独占的，因此 server 端 TCP 连接 4 元组中只有客户端的 IP 地址和端口号是可变的，因此最大 TCP 连接为客户端 IP 数 $\times$ 客户端 port 数，对 IPV4，在不考虑 IP 地址分类的情况下，最大 TCP 连接数约为 2 的 32 次方（IP 数） $\times$ 2 的 16 次方（port 数），也就是 server 端单机最大 TCP 连接数约为 2 的 48 次方。

然而上面给出的是只是理论上的单机最大连接数，在实际环境中，受到明文规定（一些 IP 地址和端口具有特殊含义，没有对外开放）、机器资源、操作系统等的限制，特别是 sever 端，其最大并发 TCP 连接数远不能达到理论上限。对 server 端，通过增加内存、修改最大文件描述符个数等参数，单机最大并发 TCP 连接数超过 10 万是没问题的。

TCP 流量控制与拥塞控制

流量控制

所谓流量控制就是让发送方的发送速率不要太快，让接收方来得及接收。如果接收方来不及接收发送方发送的数据，那么就会有分组丢失。在 TCP 中利用可变长的滑动窗口机制可以很方便的在 TCP 连接上实现对发送方的流量控制。主要的方式是接收方返回的 ACK 中会包含自己的接收窗口大小，以控制发送方此次发送的数据量大小（发送窗口大小）。

拥塞控制

在实际的网络通信系统中，除了发送方和接收方外，还有路由器，交换机等复杂的网络传输线路，此时就需要拥塞控制。拥塞控制是作用于网络的，它是防止过多的数据注入到网络中，避免出现网络负载过大的情况。常用的解决方法有：慢开始和拥塞避免、快重传和快恢复。

拥塞控制和流量控制的区别

拥塞控制往往是一种全局的，防止过多的数据注入到网络之中，而 TCP 连接的端点只要不能收到对方的确认信息，猜想在网络中发生了拥塞，但并不知道发生在何处，因此，流量控制往往指点对点通信量的控制，是端到端的问题。

如果接收方滑动窗口满了，发送方会怎么做

基于 TCP 流量控制中的滑动窗口协议，我们知道接收方返回给发送方的 ACK 包中会包含自己的接收窗口大小，若接收窗口已满，此时接收方返回给发送方的接收窗口大小为 0，此时发送方会等待接收方发送的窗口大小直到变为非 0 为止，然而，接收方回应的 ACK 包是存在丢失的可能的，为了防止双方一直等待而出现死锁情况，此时就需要坚持计时器来辅助发送方周期性地向接收方查询，以便发现窗口是否变大【坚持计时器参考问题】，当发现窗口大小变为非零时，发送方便继续发送数据。

TCP 拥塞控制采用的四种算法

- 慢开始

当发送方开始发送数据时，由于一开始不知道网络负荷情况，如果立即将大量的数据字节传输到网络中，那么就有可能引起网络拥塞。一个较好的方法是在一开始发送少量的数据先探测一下网络状况，即由小到大的增大发送窗口（拥塞窗口 cwnd）。慢开始的慢指的是初始时令 cwnd 为 1，即一开始发送一个报文段。如果收到确认，则 $cwnd = 2$ ，之后每收到一个确认报文，就令 $cwnd = cwnd * 2$ 。

但是，为了防止拥塞窗口增长过大而引起网络拥塞，另外设置了一个慢开始门限 ssthresh。

- ① 当 $cwnd < ssthresh$ 时，使用上述的慢开始算法；
- ② 当 $cwnd > ssthresh$ 时，停止使用慢开始，转而使用拥塞避免算法；
- ③ 当 $cwnd == ssthresh$ 时，两者均可。

- 拥塞避免

拥塞控制是为了让拥塞窗口 cwnd 缓慢地增大，即每经过一个往返时间 RTT（往返时间定义为发送方发送数据到收到确认报文所经历的时间）就把发送方的 cwnd 值加 1，通过让 cwnd 线性增长，防止很快就遇到网络拥塞状态。

当网络拥塞发生时，让新的慢开始门限值变为发生拥塞时候的值的一半，并将拥塞窗口置为 1，然后再次重复两种算法（慢开始和拥塞避免），这时一瞬间会将网络中的数据量大量降低。

- 快重传

快重传算法要求接收方每收到一个失序的报文就立即发送重复确认，而不要等到自己发送数据时才捎带进行确认，假定发送方发送了 Msg 1 ~ Msg 4 这 4 个报文，已知接收方收到了 Msg 1，Msg 3 和 Msg 4 报文，此时因为接收到收到了失序的数据包，按照快重传的约定，接收方应立即向发送方发送 Msg 1 的重复确认。于是在接收方收到 Msg 4 报文的时候，向发送方发送的仍然是 Msg 1 的重复确认。这样，发送方就收到了 3 次 Msg 1 的重复确认，于是立即重传对方未收到的 Msg 报文。由于发送方尽早重传未被确认的报文段，因此，快重传算法可以提高网络的吞吐量。

- 快恢复

快恢复算法是和快重传算法配合使用的，该算法主要有以下两个要点：

- ① 当发送方连续收到三个重复确认，执行乘法减小，慢开始门限 ssthresh 值减半；
- ② 由于发送方可能认为网络现在没有拥塞，因此与慢开始不同，把 cwnd 值设置为 ssthresh 减半之后的值，然后执行拥塞避免算法，线性增大 cwnd。

TCP 粘包问题

为什么会发生TCP粘包和拆包？

- ① 发送方写入的数据大于套接字缓冲区的大小，此时将发生拆包。
- ② 发送方写入的数据小于套接字缓冲区大小，由于 TCP 默认使用 Nagle 算法，只有当收到一个确认后，才将分组发送给对端，当发送方收集了多个较小的分组，就会一起发送给对端，这将会发生粘包。
- ③ 进行 MSS（最大报文长度）大小的 TCP 分段，当 TCP 报文的数据部分大于 MSS 的时候将发生拆包。
- ④ 发送方发送的数据太快，接收方处理数据的速度赶不上发送端的速度，将发生粘包。

常见解决方法

- ① 在消息的头部添加消息长度字段，服务端获取消息头的时候解析消息长度，然后向后读取相应长度的内容。
- ② 固定消息数据的长度，服务端每次读取既定长度的内容作为一条完整消息，当消息不够长时，空位补上固定字符。但是该方法会浪费网络资源。

③ 设置消息边界，也可以理解为分隔符，服务端从数据流中按消息边界分离出消息内容，一般使用换行符。

什么时候需要处理粘包问题？

当接收端同时收到多个分组，并且这些分组之间毫无关系时，需要处理粘包；而当多个分组属于同一数据的不同部分时，并不需要处理粘包问题。

TCP 报文包含哪些信息

TCP 报文是 TCP 传输的数据单元，也叫做报文段，其报文格式如下图所示：



- 源端口和目的端口号：它用于多路复用/分解来自或送往上层应用的数据，其和 IP 数据报中的源 IP 与目的 IP 地址一同确定一条 TCP 连接。
- 序号和确认号字段：序号是本报文段发送的数据部分中第一个字节的编号，在 TCP 传送的流中，每一个字节一个序号。例如一个报文段的序号为 100，此报文段数据部分共有 100 个字节，则下一个报文段的序号为 200。序号确保了 TCP 传输的有序性。确认号，即 ACK，指明下一个想要收到的字节序号，发送 ACK 时表明当前序号之前的所有数据已经正确接收。这两个字段的主要目的是保证数据可靠传输。
- 首部长度：该字段指示了以 32 比特的字为单位的 TCP 的首部长度。其中固定字段长度为 20 字节，由于首部长度可能含有可选项内容，因此 TCP 报头的长度是不确定的，20 字节是 TCP 首部的最小长度。
- 保留：为将来用于新的用途而保留。
- 控制位：URG 表示紧急指针标志，该位为 1 时表示紧急指针有效，为 0 则忽略；ACK 为确认序号标志，即相应报文段包括一个对已被成功接收报文段的确认；PSH 为 push 标志，当该位为 1 时，则指示接收方在接收到该报文段以后，应尽快将这个报文段交给应用程序，而不是在缓冲区排队；RST 为重置连接标志，当出现错误连接时，使用此标志来拒绝非法的请求；SYN 为同步序号，在连接的建立过程中使用，例如三次握手时，发送方发送 SYN 包表示请求建立连接；FIN 为 finish 标志，用于释放连接，为 1 时表示发送方已经没有数据发送了，即关闭本方数据流。
- 接收窗口：主要用于 TCP 流量控制。该字段用来告诉发送方其窗口（缓冲区）大小，以此控制发送速率，从而达到流量控制的目的。
- 校验和：奇偶校验，此校验和是对整个 TCP 报文段，包括 TCP 头部和数据部分。该校验和是一个端到端的校验和，由发送端计算和存储，并由接收端进行验证，主要目的是检验数据是否发生改动，若检测出差错，接收方会丢弃该 TCP 报文。
- 紧急数据指针：紧急数据用于告知紧急数据所在的位置，在 URG 标志位为 1 时才有效。当紧急数据存在时，TCP 必须通知接收方的上层实体，接收方会对紧急模式采取相应的处理。

- 选项：该字段一般为空，可根据首部长度进行推算。主要有以下作用：

- ① TCP 连接初始化时，通信双方确认最大报文长度。
- ② 在高速数据传输时，可使用该选项协商窗口扩大因子。
- ③ 作为时间戳时，提供一个较为精准的 RTT。

- 数据：TCP 报文中的数据部分也是可选的，例如在 TCP 三次握手和四次挥手过程中，通信双方交换的报文只包含头部信息，数据部分为空，只有当连接成功建立后，TCP 包才真正携带数据。

SYN FLOOD 是什么

SYN Flood 是种典型的 DoS（拒绝服务）攻击，其目的是通过消耗服务器所有可用资源使服务器无法用于处理合法请求。通过重复发送初始连接请求（SYN）数据包，攻击者能够压倒目标服务器上的所有可用端口，导致目标设备根本不响应合法请求。

为什么服务端易受到 SYN 攻击

在 TCP 建立连接的过程中，因为服务端不确定自己发给客户端的 SYN-ACK 消息或客户端反馈的 ACK 消息是否会丢在半路，所以会给每个待完成的半开连接状态设一个定时器，如果超过时间还没有收到客户端的 ACK 消息，则重新发送一次 SYN-ACK 消息给客户端，直到重试超过一定次数时才会放弃。

服务端为了维持半开连接状态，需要分配内核资源维护半开连接。当攻击者伪造海量的虚假 IP 向服务端发送 SYN 包时，就形成了 SYN FLOOD 攻击。攻击者故意不响应 ACK 消息，导致服务端被大量注定不能完成的半开连接占据，直到资源耗尽，停止响应正常的连接请求。

解决方法：

- 直接的方法是提高 TCP 端口容量的同时减少半开连接的资源占用时间，然而该方法只是稍稍提高了防御能力；
- 部署能够辨别恶意 IP 的路由器，将伪造 IP 地址的发送方发送的 SYN 消息过滤掉，该方案作用一般不是太大；

上述两种方法虽然在一定程度上能够提高服务器的防御能力，但是没有从根本上解决服务器资源消耗殆尽的问题，而以下几种方法的出发点都是在发送方发送确认回复后才开始分配传输资源，从而避免服务器资源消耗殆尽。

- SYN Cache：该方法首先构造一个全局 Hash Table，用来缓存系统当前所有的半开连接信息。在 Hash Table 中的每个桶的容量大小是有限制的，当桶满时，会主动丢掉早来的信息。当服务端收到一个 SYN 消息后，会通过一个映射函数生成一个相应的 Key 值，使得当前半连接信息存入相应的桶中。当收到客户端正确的确认报文后，服务端才开始分配传输资源块，并将相应的半开连接信息从表中删除。和服务器传输资源相比，维护表的开销要小得多。
- SYN Cookies：该方案原理和 HTTP Cookies 技术类似，服务端通过特定的算法将半开连接信息编码成序列号或者时间戳，用作服务端给客户端的消息编号，随 SYN-ACK 消息一同返回给连接发起方，这样在连接建立完成前服务端不保存任何信息，直到发送方发送 ACK 确认报文并且服务端成功验证编码信息后，服务端才开始分配传输资源。若请求方是攻击者，则不会向服务端发 ACK 消息，由于未成功建立连接，因此服务端并没有花费任何额外的开销。

然而该方案也存在一些缺点，由于服务端并不保存半开连接状态，因此也就丧失了超时重传的能力，这在一定程度上降低了正常用户的连接成功率。此外，客户端发送给服务端的确认报文存在传输丢失的可能，当 ACK 确认报文丢失时，服务端和客户端会对连接的成功与否产生歧义，此时就需要上层应用采取相应的策略进行了。

- SYN Proxy：在客户端和服务端之间部署一个代理服务器，类似于防火墙的作用。通过代理服务器与客户端进行建立连接的过程，之后代理服务器充当客户端将成功建立连接的客户端信息发送给服务器。这种方法基本不消耗服务器的资源，但是建立连接的时间变长了（总共需要 6 次握手）。

高并发服务器客户端主动关闭连接和服务端主动关闭连接的区别

以下是针对 TCP 服务来说的：

- **服务端主动关闭连接**

在高并发场景下，当服务端主动关闭连接时，此时服务器上就会有大量的连接处于 TIME-WAIT 状态【详见问题 7, 8, 9】

- **客户端主动关闭连接**

当客户端主动关闭连接时，我们并不需要关心 TIME-WAIT 状态过多造成的问题，但是需要关注服务端保持大量的 CLOSE-WAIT 状态时会产生问题【见问题 10 的解决方法】

无论是客户端还是服务器主动关闭连接，从本质上来说，在高并发场景下主要关心的就是服务端的资源占用问题，而这也是采用 TCP 传输协议必须要面对的问题，其问题解决的出发点也是如何处理好服务质量和资源消耗之间的关系。

第四部分：网络层

IP 协议的定义和作用

IP 协议（Internet Protocol）又称互联网协议，是支持网间互联的数据包协议。该协议工作在网络层，主要目的就是为了提高网络的可扩展性，和传输层 TCP 相比，IP 协议提供一种无连接/不可靠、尽力而为的数据包传输服务，其与 TCP 协议（传输控制协议）一起构成了 TCP/IP 协议族的核心。IP 协议主要有以下几个作用：

- 寻址和路由：在 IP 数据包中会携带源 IP 地址和目的 IP 地址来标识该数据包的源主机和目的主机。IP 数据报在传输过程中，每个中间节点（IP 网关、路由器）只根据网络地址进行转发，如果中间节点是路由器，则路由器会根据路由表选择合适的路径。IP 协议根据路由选择协议提供的路由信息对 IP 数据报进行转发，直至抵达目的主机。
- 分段与重组：IP 数据包在传输过程中可能会经过不同的网络，在不同的网络中数据包的最大长度限制是不同的，IP 协议通过给每个 IP 数据包分配一个标识符以及分段与组装的相关信息，使得数据包在不同的网络中能够传输，被分段后的 IP 数据报可以独立地在网络中进行转发，在到达目的主机后由目的主机完成重组工作，恢复出原来的 IP 数据包。

域名和 IP 的关系，一个 IP 可以对应多个域名吗

整个互联网是一个单一的、抽象的网络。互联网上每一台主机或者路由器在连入互联网时需要被分配一个唯一的标识符，即 IP 地址，在 IPv4 协议下，这个标识符是一个 32 位无符号二进制数。互联网上和一个设备通信时，需要知道其 IP 地址。互联网最常见的应用之一是向某个特定主机请求一些服务，例如请求某个网页的信息，请求某个邮件服务器的服务。然而，数字 IP 地址不方便人们记忆和描述。为此，域名系统（Domain Name System，DNS）被开发和应用，使得人们可以方便地将一串便于记忆和描述的字符串，称为域名，利用 DNS 自动转换为 IP 地址。互联网上分布运行着许多域名服务器，其他互联网上的设备可以向它们询问域名对应的 IP 地址是什么，此即域名解析。网站的运营者可以在域名注册商处注册域名，使其指向自己的网站服务器的 IP 地址，使得这一域名被其他互联网上的用户使用。

在最简单的情形下，一个域名只对应一个 IP 地址，一个 IP 地址只对应一个域名。

实际中，根据需求，多个域名可以被解析为同一个 IP 地址。例如具有一个 IP 地址的服务器上的运营的网站，为了推广自己给自己注册了许多朗朗上口的域名；或者提前注册多个类似域名，防止被竞争者不当利用；或者一个网站历史上使用了另一个域名，现在多个域名同时可用，并被解析到同一个 IP 地址。此外，相对少见地，一个域名可能会被解析出多个 IP 地址。例如，这个网站为了平衡负载，在多个服务

器上提供了同一类服务。这可能表现为：不同时间，同一个域名被解析为不同的 IP；一个域名解析的结果可能有若干备选 IP；一个域名不同的网络服务提供商的网络中会被解析出不同的 IP。

示例：如图，在实现了 Posix 的计算机系统上，nslookup 命令可以用于解析域名。运行 nslookup www.baidu.com 可能会返回多个 IP 地址，而查询 www.baidu.com 和 www.a.shifen.com 的 IP 地址包含了重复的 IP 地址条目。

参考资料：RFC 791，RFC 1034，1035。

```
~ % date
2021年 9月24日 星期五 16时32分36秒 CST
~ % nslookup www.baidu.com
Server:          223.5.5.5
Address:         223.5.5.5#53

Non-authoritative answer:
www.baidu.com    canonical name = www.a.shifen.com.
Name:   www.a.shifen.com
Address: 180.101.49.11
Name:   www.a.shifen.com
Address: 180.101.49.12

~ % nslookup www.a.shifen.com
Server:          223.5.5.5
Address:         223.5.5.5#53

Non-authoritative answer:
Name:   www.a.shifen.com
Address: 180.101.49.12
Name:   www.a.shifen.com
Address: 180.101.49.11
```

IPV4 地址不够如何解决

- DHCP：动态主机配置协议。动态分配 IP 地址，只给接入网络的设备分配IP地址，因此同一个 MAC 地址的设备，每次接入互联网时，得到的IP地址不一定是相同的，该协议使得空闲的 IP 地址可以得到充分利用。
- CIDR：无类别域间路由。CIDR 消除了传统的 A 类、B 类、C 类地址以及划分子网的概念，因而更加有效的分配 IPv4 的地址空间，但无法从根本上解决地址耗尽问题。
- NAT：网络地址转换协议。我们知道属于不同局域网的主机可以使用相同的 IP 地址，从而一定程度上缓解了 IP 资源枯竭的问题。然而主机在局域网中使用的 IP 地址是不能在公网中使用的，当局域网主机想要与公网进行通信时，NAT 方法可以将该主机 IP 地址转换成全球 IP 地址。该协议能够有效解决 IP 地址不足的问题。
- IPv6：作为接替 IPv4 的下一代互联网协议，其可以实现 2^{128} 次方个地址，而这个数量级，即使是给地球上每一颗沙子都分配一个IP地址，该协议能够从根本上解决 IPv4 地址不够用的问题。

路由器的分组转发流程

- ① 从 IP 数据包中提取出目的主机的 IP 地址，找到其所在的网络；
- ② 判断目的 IP 地址所在的网络是否与本路由器直接相连，如果是，则不需要经过其它路由器直接交付，否则执行 ③；
- ③ 检查路由表中是否有目的 IP 地址的特定主机路由。如果有，则按照路由表传送到下一跳路由器中，否则执行 ④；
- ④ 逐条检查路由表，使用每一行的子网掩码与目的IP匹配。若找到匹配路由，则按照路由表转发到下一跳路由器中，否则执行步骤 ⑤；
- ⑤ 若路由表中设置有默认路由，则按照默认路由转发到默认路由器中，否则执行步骤 ⑥；

⑥ 无法找到合适路由，向源主机报错。

路由器和交换机的区别

- 交换机：交换机用于局域网，利用主机的物理地址（MAC 地址）确定数据转发的目的地址，它工作于数据链路层。
- 路由器：路由器通过数据包中的目的 IP 地址识别不同的网络从而确定数据转发的目的地址，网络号是唯一的。路由器根据路由选择协议和路由表信息从而确定数据的转发路径，直到到达目的网络，它工作于网络层。

ICMP 协议概念/作用

ICMP (Internet Control Message Protocol) 是因特网控制报文协议，主要是实现 IP 协议中未实现的部分功能，是一种网络层协议。该协议并不传输数据，只传输控制信息来辅助网络层通信。其主要的功能是验证网络是否畅通（确认接收方是否成功接收到 IP 数据包）以及辅助 IP 协议实现可靠传输（若发生 IP 丢包，ICMP 会通知发送方 IP 数据包被丢弃的原因，之后发送方会进行相应的处理）。

ICMP 的应用

- Ping
Ping (Packet Internet Groper)，即因特网包探测器，是一种工作在网络层的服务命令，主要用于测试网络连接量。本地主机通过向目的主机发送 ICMP Echo 请求报文，目的主机收到之后会发送 Echo 响应报文，Ping 会根据时间和成功响应的次数估算出数据包往返时间以及丢包率从而推断网络是否通畅、运行是否正常等。
- TraceRoute
TraceRoute 是 ICMP 的另一个应用，其主要用来跟踪一个分组从源点耗费最少 TTL 到达目的地的路径。TraceRoute 通过逐渐增大 TTL 值并重复发送数据报来实现其功能，首先，TraceRoute 会发送一个 TTL 为 1 的 IP 数据报到目的地，当路径上的第一个路由器收到这个数据报时，它将 TTL 的值减 1，此时 $TTL = 0$ ，所以路由器会将这个数据报丢掉，并返回一个差错报告报文，之后源主机接着发送一个 TTL 为 2 的数据报，并重复此过程，直到数据报能够刚好到达目的主机。此时 $TTL = 0$ ，因此目的主机要向源主机发送 ICMP 终点不可达差错报告报文，之后源主机便知道了到达目的主机所经过的路由器 IP 地址以及到达每个路由器的往返时间。

两台电脑连起来后 ping 不通，你觉得可能存在哪些问题？

- 首先看网络是否连接正常，检查网卡驱动是否正确安装。
- 局域网设置问题，检查 IP 地址是否设置正确。
- 看是否被防火墙阻拦（有些设置中防火墙会对 ICMP 报文进行过滤），如果是的话，尝试关闭防火墙。
- 看是否被第三方软件拦截。
- 两台设备间的网络延迟是否过大（例如路由设置不合理），导致 ICMP 报文无法在规定的时间内收到。

ARP 地址解析协议的原理和地址解析过程

ARP (Address Resolution Protocol) 是地址解析协议的缩写，该协议提供根据 IP 地址获取物理地址的功能，它工作在第二层，是一个数据链路层协议，其在本层和物理层进行联系，同时向上层提供服务。当通过以太网发送 IP 数据包时，需要先封装 32 位的 IP 地址和 48 位 MAC 地址。在局域网中两台主机进行通信时需要依靠各自的物理地址进行标识，但由于发送方只知道目标 IP 地址，不知道其 MAC 地址，因此需要使用地址解析协议。ARP 协议的解析过程如下：

① 首先，每个主机都会在自己的 ARP 缓冲区中建立一个 ARP 列表，以表示 IP 地址和 MAC 地址之间的对应关系；

② 当源主机要发送数据时，首先检查 ARP 列表中是否有 IP 地址对应的目的主机 MAC 地址，如果存在，则可以直接发送数据，否则就向同一子网的所有主机发送 ARP 数据包。该数据包包括的内容有源主机的 IP 地址和 MAC 地址，以及目的主机的 IP 地址。

③ 当本网络中的所有主机收到该 ARP 数据包时，首先检查数据包中的 目的 主机IP 地址是否是自己的 IP 地址，如果不是，则忽略该数据包，如果是，则首先从数据包中取出源主机的 IP 和 MAC 地址写入到 ARP 列表中，如果已经存在，则覆盖，然后将自己的 MAC 地址写入 ARP 响应包中，告诉源主机自己是它要找的 MAC 地址。

④ 源主机收到 ARP 响应包后。将目的主机的 IP 和 MAC 地址写入 ARP 列表，并利用此信息发送数据。如果源主机一直没有收到 ARP 响应数据包，表示 ARP 查询失败。

网络地址转换 NAT

NAT (Network Address Translation) ，即网络地址转换，它是一种把内部私有网络地址翻译成公有网络 IP 地址的技术。该技术不仅能解决 IP 地址不足的问题，而且还能隐藏和保护网络内部主机，从而避免来自外部网络的攻击。

NAT 的实现方式主要有三种：

- 静态转换：内部私有 IP 地址和公有 IP 地址是一一对应的关系，并且不会发生改变。通过静态转换，可以实现外部网络对内部网络特定设备的访问，这种方式原理简单，但当某一共有 IP 地址被占用时，跟这个 IP 绑定的内部主机将无法访问 Internet。
- 动态转换：采用动态转换的方式时，私有 IP 地址每次转化成的公有 IP 地址是不唯一的。当私有 IP 地址被授权访问 Internet 时会被随机转换成一个合法的公有 IP 地址。当 ISP 通过的合法 IP 地址数量略少于网络内部计算机数量时，可以采用这种方式。
- 端口多路复用：该方式将外出数据包的源端口进行端口转换，通过端口多路复用的方式，实现内部网络所有主机共享一个合法的外部 IP 地址进行 Internet 访问，从而最大限度地节约 IP 地址资源。同时，该方案可以隐藏内部网络中的主机，从而有效避免来自 Internet 的攻击。

TTL 是什么？有什么作用

TTL 是指生存时间，简单来说，它表示了数据包在网络中的时间。每经过一个路由器后 TTL 就减一，这样 TTL 最终会减为 0，当 TTL 为 0 时，则将数据包丢弃。通过设置 TTL 可以避免这两个路由器之间形成环导致数据包在环路上死转的情况，由于有了 TTL，当 TTL 为 0 时，数据包就会被抛弃。

运输层协议和网络层协议的区别

网络层协议负责提供主机间的逻辑通信；运输层协议负责提供进程间的逻辑通信。

第五部分：数据链路层

MAC 地址和 IP 地址分别有什么作用

- MAC 地址是数据链路层和物理层使用的地址，是写在网卡上的物理地址。MAC 地址用来定义网络设备的位置。
- IP 地址是网络层和以上各层使用的地址，是一种逻辑地址。IP 地址用来区别网络上的计算机。

为什么有了 MAC 地址还需要 IP 地址

如果我们只使用 MAC 地址进行寻址的话，我们需要路由器记住每个 MAC 地址属于哪一个子网，不然每一次路由器收到数据包时都要满世界寻找目的 MAC 地址。而我们知道 MAC 地址的长度为 48 位，也就是说最多总共有 2^{48} 个 MAC 地址，这就意味着每个路由器需要 256 T 的内存，这显然是不现实的。

和 MAC 地址不同，IP 地址是和地域相关的，在一个子网中的设备，我们给其分配的 IP 地址前缀都是一样的，这样路由器就能根据 IP 地址的前缀知道这个设备属于哪个子网，剩下的寻址就交给子网内部实现，从而大大减少了路由器所需要的内存。

为什么有了 IP 地址还需要 MAC 地址

只有当设备连入网络时，才能根据他进入了哪个子网来为其分配 IP 地址，在设备还没有 IP 地址的时候或者在分配 IP 地址的过程中，我们需要 MAC 地址来区分不同的设备。

私网地址和公网地址之间进行转换：同一个局域网内的两个私网地址，经过转换之后外面看到的一样吗

当采用静态或者动态转换时，由于一个私网 IP 地址对应一个公网地址，因此经过转换之后的公网 IP 地址是不同的；而采用端口复用方式的话，在一个子网中的所有地址都采用一个公网地址，但是使用的端口是不同的。

以太网中的 CSMA/CD 协议

CSMA/CD 为载波侦听多路访问/冲突检测，是像以太网这种广播网络采用的一种机制，我们知道在以太网中多台主机在同一个信道中进行数据传输，CSMA/CD 很好的解决了共享信道通信中出现的问题，它的工作原理主要包括两个部分：

- 载波监听：当使用 CSMA/CD 协议时，总线上的各个节点都在监听信道上是否有信号在传输，如果有的话，表明信道处于忙碌状态，继续保持监听，直到信道空闲为止。如果发现信道是空闲的，就立即发送数据。
- 冲突检测：当两个或两个以上节点同时监听到信道空闲，便开始发送数据，此时就会发生碰撞（数据的传输延迟也可能引发碰撞）。当两个帧发生冲突时，数据帧就会破坏而失去了继续传输的意义。在数据的发送过程中，以太网是一直在监听信道的，当检测到当前信道冲突，就立即停止这次传输，避免造成网络资源浪费，同时向信道发送一个「冲突」信号，确保其它节点也发现该冲突。之后采用一种二进制退避策略让待发送数据的节点随机退避一段时间之后重新。

数据链路层上的三个基本问题

- 封装成帧：将网络层传下来的分组前后分别添加首部和尾部，这样就构成了帧。首部和尾部的一个重要作用是帧定界，也携带了一些必要的控制信息，对于每种数据链路层协议都规定了帧的数据部分的最大长度。
- 透明传输：帧使用首部和尾部进行定界，如果帧的数据部分含有和首部和尾部相同的内容，那么帧的开始和结束的位置就会判断错，因此需要在数据部分中出现有歧义的内容前边插入转义字符，如果数据部分出现转义字符，则在该转义字符前再加一个转义字符。在接收端进行处理之后可以还原出原始数据。这个过程透明传输的内容是转义字符，用户察觉不到转义字符的存在。
- 差错检测：目前数据链路层广泛使用循环冗余检验（CRC）来检查数据传输过程中是否产生比特差错。

PPP 协议

互联网用户通常需要连接到某个 ISP 之后才能接入到互联网，PPP（点对点）协议是用户计算机和 ISP 进行通信时所使用的数据链路层协议。点对点协议为点对点连接上传输多协议数据包提供了一个标准方法。该协议设计的目的是用来通过拨号或专线方式建立点对点连接发送数据，使其成为各种主机、网桥和路由器之间简单连接的一种解决方案。

PPP 协议具有以下特点：

- PPP 协议具有动态分配 IP 地址的能力，其允许在连接时刻协商 IP 地址。
- PPP 支持多种网络协议，例如 TCP/IP、NETBEUI 等。

- PPP 具有差错检测能力，但不具备纠错能力，所以 PPP 是不可靠传输协议。
- 无重传的机制，网络开销小，速度快。
- PPP 具有身份验证的功能。

为什么 PPP 协议不使用序号和确认机制

- IETF 在设计因特网体系结构时把其中最复杂的部分放在 TCP 协议中，而网际协议 IP 则相对比较简单，它提供的是不可靠的数据包服务，在这种情况下，数据链路层没有必要提供比 IP 协议更多的功能。若使用能够实现可靠传输的数据链路层协议，则开销就要增大，这在数据链路层出现差错概率不大时是得不偿失的。
- 即使数据链路层实现了可靠传输，但其也不能保证网络层的传输也是可靠的，当数据帧在路由器中从数据链路层上升到网络层后，仍有可能因为网络层拥塞而被丢弃。
- PPP 协议在帧格式中有帧检验序列，对每一个收到的帧，PPP 都会进行差错检测，若发现差错，则丢弃该帧。

第六部分：物理层

物理层主要做什么事情

作为 OSI 参考模型最低的一层，物理层是整个开放系统的基础，该层利用传输介质为通信的两端建立、管理和释放物理连接，实现比特流的透明传输。物理层考虑的是怎样才能在连接各种计算机的传输媒体上传输数据比特流，其尽可能地屏蔽掉不同种类传输媒体和通信手段的差异，使物理层上面的数据链路层感觉不到这些差异，这样就可以使数据链路层只考虑完成本层的协议和服务，而不必考虑网络的具体传输媒体和通信手段是什么。

主机之间的通信方式

- 单工通信：也叫单向通信，发送方和接收方是固定的，消息只能单向传输。例如采集气象数据、家庭电费，网费等数据收集系统，或者打印机等应用主要采用单工通信。
- 半双工通信：也叫双向交替通信，通信双方都可以发送消息，但同一时刻同一信道只允许单方向发送数据。例如传统的对讲机使用的就是半双工通信。
- 全双工通信：也叫双向同时通信，全双工通信允许通信双方同时在两个方向上传输，其要求通信双方都具有独立的发送和接收数据的能力。例如平时我们打电话，自己说话的同时也能听到对面的声音。

通道复用技术

- 频分复用 (FDM, Frequency Division Multiplexing)

频分复用将传输信道的总带宽按频率划分为若干个子频带或子信道，每个子信道传输一路信号。用户分到一定的频带后，在数据传输的过程中自始至终地占用这个频带。由于每个用户所分到的频带不同，使得传输信道在同一时刻能够支持不同用户进行数据传输，从而实现复用。除了传统意义上的 FDM 外，目前正交频分复用 (OFDM) 已在高速通信系统中得到广泛应用。

- 时分复用 (TDM, Time Division Multiplexing)

顾名思义，时分复用将信道传输信息的时间划分为若干个时间片，每一个时分复用的用户在每一个 TDM 帧中占用固定时隙进行数据传输。用户所分配到的时隙是固定的，所以时分复用有时也叫做同步时分复用。这种分配方式能够便于调节控制，但是也存在缺点，当某个信道空闲时，其他繁忙的信道无法占用该空闲信道，因此会降低信道利用率。

- 波分复用 (WDM, Wavelength Division Multiplexing)

在光通信领域通常按照波长而不是频率来命名，因为光的频率和波长具有单一对应关系，因此 WDM 本质上也是 FDM，光通信系统中，通常由光来运载信号进行传输，WDM 是在一条光纤上传输多个波长光信号，其将 1 根光纤看做多条「虚拟」光纤，每条「虚拟」光纤工作在不同的波长上，从而极大地提高了光纤的传输容量。

- 码分复用（CDM，Code Division Multiplexing）

码分复用是靠不同的编码来区分各路原始信号的一种复用方式，不同的用户使用相互正交的码字携带信息。由于码组相互正交，因此接收方能够有效区分不同的用户数据，从而实现每一个用户可以在同样的时间在同样的频带进行数据传输，频谱资源利用率高。其主要和各种多址接入技术相结合从而产生各种接入技术，包括无线和优先接入。

几种常用的宽带接入技术

我们一般将速率超过 1 Mbps 的接入称为宽带接入，目前常用的宽带接入技术主要包括：ADSL 和 FTTx + LAN。

- ADSL

ADSL 全称为非对称用户数字环路，是铜线宽带接入技术的一种。其非对称体现在用户上行和下行的传输速率不相等，一般上行速率较低，下行速率高。这种接入技术适用于有宽带业务需求的家庭用户或者中小型商务用户等。

- FTTx + LAN

其中 FTTx 英文翻译为 Fiber To The X，这里的 X 指任何地方，我们可以理解为光纤可以接入到任何地方，而 LAN 指的是局域网。FTTx + LAN 是一种在接入网全部或部分采用光纤传输介质，构成光纤用户线路，从而实现用户高速上网的接入技术，其中用户速率可达 20 Mbps。这种接入技术投资规模小，网络拓展性强，网络可靠稳定，使得其应用广泛，目前是城市汇总较为普及的一种宽带接入技术。

其它还有 光纤同轴混合网（HFC）、光接入技术（有源和无源光纤系统）和无线接入技术等等。

第七部分：计算机网络中的安全

安全攻击有哪些

网络安全攻击主要分为被动攻击和主动攻击两类：

- 被动攻击：攻击者窃听和监听数据传输，从而获取到传输的数据信息，被动攻击主要有两种形式：消息内容泄露攻击和流量分析攻击。由于攻击者并没有修改数据，使得这种攻击类型是很难被检测到的。
- 主动攻击：攻击者修改传输的数据流或者故意添加错误的数据流，例如假冒用户身份从而得到一些权限，进行权限攻击，除此之外，还有重放、改写和拒绝服务等主动攻击的方式。

ARP 攻击

在 ARP 的解析过程中，局域网上的任何一台主机如果接收到一个 ARP 应答报文，并不会去检测这个报文的真实性，而是直接记入自己的 ARP 缓存表中。并且这个 ARP 表是可以被更改的，当表中的某一列长时间不适使用，就会被删除。ARP 攻击就是利用了这一点，攻击者疯狂发送 ARP 报文，其源 MAC 地址为攻击者的 MAC 地址，而源 IP 地址为被攻击者的 IP 地址。通过不断发送这些伪造的 ARP 报文，让网络内部的所有主机和网关的 ARP 表中被攻击者的 IP 地址所对应的 MAC 地址为攻击者的 MAC 地址。这样所有发送给被攻击者的信息都会发送到攻击者的主机上，从而产生 ARP 欺骗。通常可以把 ARP 欺骗分为以下几种：

- 洪泛攻击

攻击者恶意向局域网中的网关、路由器和交换机等发送大量 ARP 报文，设备的 CPU 忙于处理 ARP 协议，而导致难以响应正常的服务请求。其表现通常为：网络中断或者网速很慢。

- 欺骗主机

这种攻击方式也叫仿冒网关攻击。攻击者通过 ARP 欺骗使得网络内部被攻击主机发送给网关的信息实际上都发送给了攻击者，主机更新的 ARP 表中对应的 MAC 地址为攻击者的 MAC。当用户主机向网关发送重要信息时，该攻击方式使得用户的数据存在被窃取的风险。

- 欺骗网关

该攻击方式和欺骗主机的攻击方式类似，不过这种攻击的欺骗对象是局域网的网关，当局域网中的主机向网关发送数据时，网关会把数据发送给攻击者，这样攻击者就会源源不断地获得局域网中用户的信息。该攻击方式同样会造成用户数据外泄。

- 中间人攻击

攻击者同时欺骗网关和主机，局域网的网关和主机发送的数据最后都会到达攻击者这边。这样，网关和用户的数据就会泄露。

- IP 地址冲突

攻击者对局域网中的主机进行扫描，然后根据物理主机的 MAC 地址进行攻击，导致局域网内的主机产生 IP 冲突，使得用户的网络无法正常使用。

对称加密和非对称的区别，非对称加密有哪些

- 加密和解密的过程不同：对称加密和解密过程使用同一个密钥；非对称加密中加密和解密采用公钥和私钥两个密钥，一般使用公钥进行加密，使用私钥进行解密。
- 加密和解密的速度不同：对称加密和解密速度较快，当数据量比较大时适合使用；非对称加密和解密时间较长，速度相对较慢，适合少量数据传输的场景。
- 传输的安全性不同：采用对称加密方式进行通信时，收发双方在数据传送前需要协定好密钥，而这个密钥还有可能被第三方窃听到的，一旦密钥泄漏，之后的通信就完全暴露给攻击者了；非对称加密采用公钥加密和私钥解密的方式，其中私钥是基于不同的算法生成的随机数，公钥可以通过私钥通过一定的算法推导得出，并且私钥到公钥的推导过程是不可逆的，也就是说公钥无法反推导出私钥，即使攻击者窃听到传输的公钥，也无法正确解出数据，所以安全性较高。

常见的非对称加密算法主要有：RSA、Elgamal、背包算法、Rabin、D-H 算法等等。

AES 的过程

AES (Advanced Encryption Standard) 即密码学的高级加密标准，也叫做 Rijndael 加密法，是为最为常见的一种对称加密算法，和传统的对称加密算法大致的流程类似，在发送端需要采用加密算法对明文进行加密，在接收端需要采用与加密算法相同的算法进行解密，不同的是，AES 采用分组加密的方式，将明文分成一组一组的，每组长度相等，每次加密一组数据，直到加密完整个明文。在 AES 标准中，分组长度固定为 128 位，即每个分组为 16 个字节（每个字节有 8 位）。而密钥的长度可以是 128 位，192 位或者 256 位。并且密钥的长度不同，推荐加密的轮数也不同。

我们以 128 位密钥为例（加密轮次为 10），已知明文首先需要分组，每一组大小为 16 个字节并形成 4×4 的状态矩阵（矩阵中的每一个元素代表一个字节）。类似地，128 位密钥同样用 4×4 的字节矩阵表示，矩阵中的每一列称为 1 个 32 位的比特字。通过密钥编排函数该密钥矩阵被扩展成一个由 44 个字组成的序列，该序列的前四个字是原始密钥，用于 AES 的初始密钥加过程，后面 40 个字分为 10 组，每组 4 个字分别用于 10 轮加密运算中的轮密钥加。在每轮加密过程中主要包括四个步骤：

① 字节代换：AES 的字节代换其实是一个简易的查表操作，在 AES 中定义了一个 S-box 和一个逆 S-box，我们可以将其简单地理解为两个映射表，在做字节代换时，状态矩阵中的每一个元素（字节）的高四位作为行值，低四位作为列值，取出 S-box 或者逆 S-box 中对应的行或者列作为输出。

② 行位移：顾名思义，就是对状态矩阵的每一行进行位移操作，其中状态矩阵的第 0 行左移 0 位，第 1 行左移 1 位，以此类推。

③ 列混合：列混合变换是通过矩阵相乘来实现的，经唯一后的状态矩阵与固定的矩阵相乘，从而得到混淆后的状态矩阵。其中矩阵相乘中涉及到的加法等价于两个字节的异或运算，而乘法相对复杂一些，对于状态矩阵中的每一个 8 位二进制数来说，首先将其与 00000010 相乘，其等效为将 8 位二进制数左移一位，若原二进制数的最高位是 1 的话再将左移后的数与 00011011 进行异或运算。

④ 轮密相加：在开始时我们提到，128 位密钥通过密钥编排函数被扩展成 44 个字组成的序列，其中前 4 个字用于加密过程开始时对原始明文矩阵进行异或运算，而后 40 个字中每四个一组在每一轮中与状态矩阵进行异或运算（共计 10 轮）。

上述过程即为 AES 加密算法的主要流程，在我们的例子中，上述过程需要经过 10 轮迭代。而 AES 的解密过程的各个步骤和加密过程是一样的，只是用逆变换取代原来的变换。

RSA 和 AES 算法有什么区别

- RSA

采用非对称加密的方式，采用公钥进行加密，私钥解密的形式。其私钥长度一般较长，除此之外，由于需要大数的乘幂求模等运算，其运算速度较慢，不适合大量数据文件加密。

- AES

采用对称加密的方式，其密钥长度最长只有 256 个比特，加密和解密速度较快，易于硬件实现。由于是对称加密，通信双方在进行数据传输前需要获知加密密钥。

基于上述两种算法的特点，一般使用 RSA 传输密钥给对方，之后使用 AES 进行加密通信。

DDoS 有哪些，如何防范

DDoS 为分布式拒绝服务攻击，是指处于不同位置的多个攻击者同时向一个或数个目标发动攻击，或者一个攻击者控制了不同位置上的多台机器并利用这些机器对受害者同时实施攻击。和单一的 DoS 攻击相比，DDoS 是借助数百台或者数千台已被入侵并添加了攻击进程的主机一起发起网络攻击。

DDoS 攻击主要有两种形式：流量攻击和资源耗尽攻击。前者主要针对网络带宽，攻击者和已受害主机同时发起大量攻击导致网络带宽被阻塞，从而淹没合法的网络数据包；后者主要针对服务器进行攻击，大量的攻击包会使得服务器资源耗尽或者 CPU 被内核应用程序占满从而无法提供网络服务。

常见的 DDos 攻击主要有：TCP 洪水攻击（SYN Flood）、放射性攻击（DrDos）、CC 攻击（HTTP Flood）等。

针对 DDoS 中的流量攻击，最直接的方法是增加带宽，理论上只要带宽大于攻击流量就可以了，但是这种方法成本非常高。在有充足网络带宽的前提下，我们应尽量提升路由器、网卡、交换机等硬件设施的配置。

针对资源耗尽攻击，我们可以升级主机服务器硬件，在网络带宽得到保证的前提下，使得服务器能有效对抗海量的 SYN 攻击包。我们也可以安装专业的抗 DDoS 防火墙，从而对抗 SYN Flood 等流量型攻击。此外，负载均衡，CDN 等技术都能够有效对抗 DDoS 攻击